

Unit 1

Hive Architecture, Hive QL

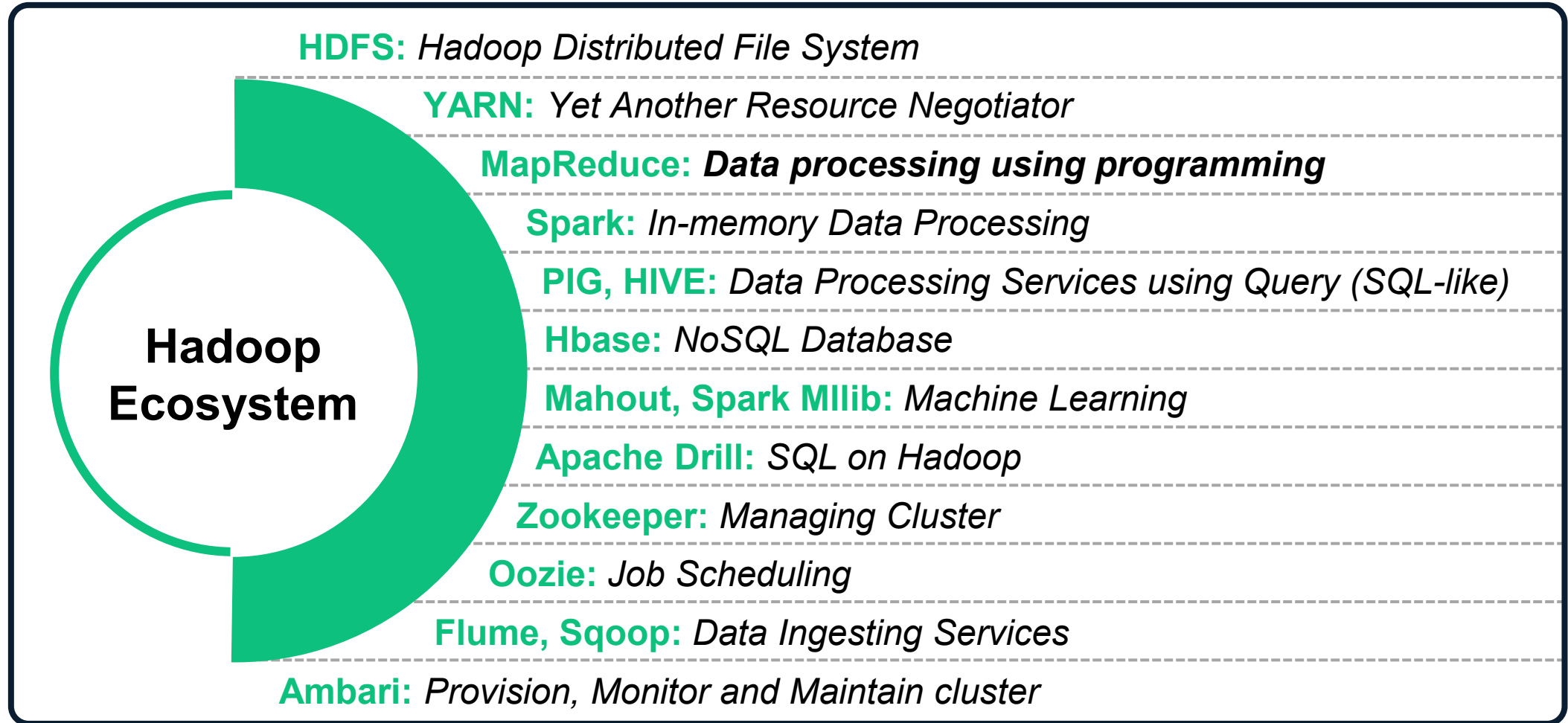
Module Topics

Let us take a quick look at the topics that we will cover in this module:

1. What is Hive?
2. History and Versions of Hive
3. What are Hive Components and its architecture?
4. How Hive is Different from Traditional DBs?
5. Hive Metastore
6. Limitations of Hive
7. Hive data types and data models, tables, partitions, buckets



1.1 Introduction to Hadoop Ecosystem



2.1 Introduction to Hive

What is Hive?

- It is a query engine wrapper built on top of MapReduce.
- It is treated as Data warehousing tool of Hadoop Ecosystem.
- Primarily targeted for SQL background users.
- Provides HiveQL, which is similar to SQL.

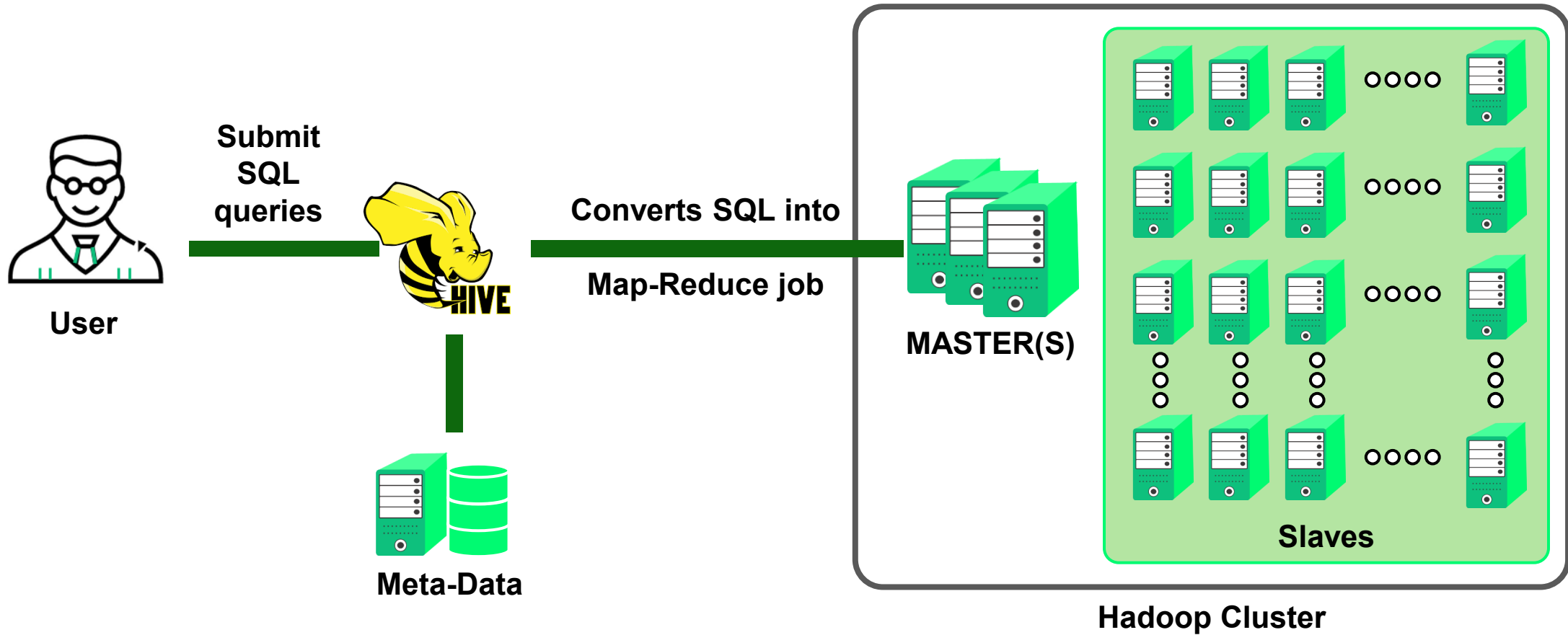
History of Apache Hive

- Developed by Facebook
- ASF developed further
- Open Source

Versions Released

- 4.2.0 Latest

2.1 Introduction to Hive (Contd.)



2.1.1 Hive vs PIG

The following table provides a comparison between Hive and PIG.

Hive	PIG
Declarative SQL based Language	Procedural Data Flow Language
For data analysis and creating reports	For Programming
Mainly used by Data Analysts	Mainly used by Researchers and Programmers
Operates on the server side of a cluster.	Operates on the client side of a cluster.
Makes use of exact variation of dedicated SQL DDL language by defining tables beforehand.	Does not have a dedicated metadata database.
Directly leverages SQL and is easy to learn for database experts.	Pig is SQL like but varies to a great extent.

2.1.2 Features of Hive

Following are the key features of Hive:

- Schema flexibility and evolution
- Tables can be partitioned and bucketed
- Apache Hive tables are defined directly in the Hadoop file system
- JDBC/ODBC drivers are available
- Fast and scalable
- Easy to learn
- Provides summarization, analysis of data and query of data at ease

Other Features of Apache Hive

- Hive feature of creating external tables, which make it feasible to process data without actually storing in HDFS.
- Apache Hive provides the low-level interface requirement of Hadoop.
- It also supports partitioning of data and bucketing and indexing at the level of tables to improve performance.
- Hive has a rule-based optimizer for optimizing execution plans.

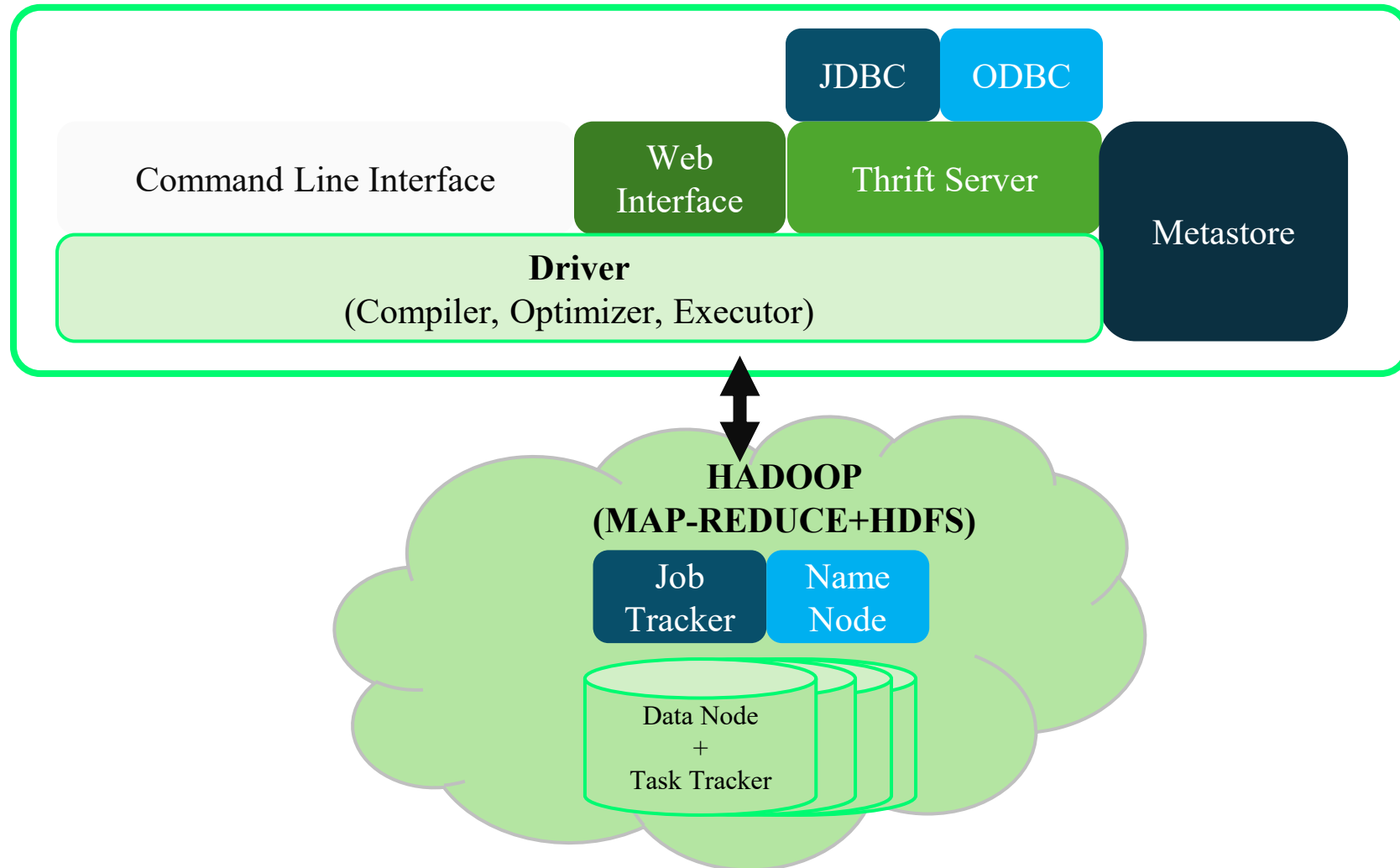
What did You Grasp?

Hive is initially developed by?

- a. Yahoo**
- b. Apache**
- c. Facebook**
- d. Google**

2.2 Hive Architecture

The picture shows the overall architecture of Hive.



Major components of the Hive architecture

Metastore:

- Stores metadata for each of the tables like table schema and location.
- It also includes the partition metadata which helps the driver to track the progress of various data sets distributed over the cluster.
- The data is stored in a traditional RDBMS format.
- The metadata helps the driver to keep track of the data.
- A backup server regularly replicates the data which can be retrieved in case of data loss.

Driver:

- Acts like a controller, which receives the HiveQL statements.
- It starts the execution of the statement by creating sessions, and monitors the life cycle and progress of the execution.
- It stores the necessary metadata generated during the execution of a HiveQL statement.
- The driver also acts as a collection point of data or query results obtained after the Reduce operation.

Major components of the Hive architecture

Compiler:

- Performs compilation of the HiveQL query, which converts the query to an execution plan.
- This plan contains the tasks and steps needed to be performed by the Hadoop MapReduce to get the output as translated by the query.
- The compiler converts the query to an abstract syntax tree (AST).
- After checking for compatibility and compile time errors, it converts the AST to a directed acyclic graph (DAG).
- The DAG divides operators to MapReduce stages and tasks based on the input query and data.

Optimizer:

- Performs various transformations on the execution plan to get an optimized DAG.
- Transformations can be aggregated together, such as converting a pipeline of joins to a single join, for better performance.
- It can also split the tasks, such as applying a transformation on data before a reduce operation, to provide better performance and scalability.

Major components of the Hive architecture

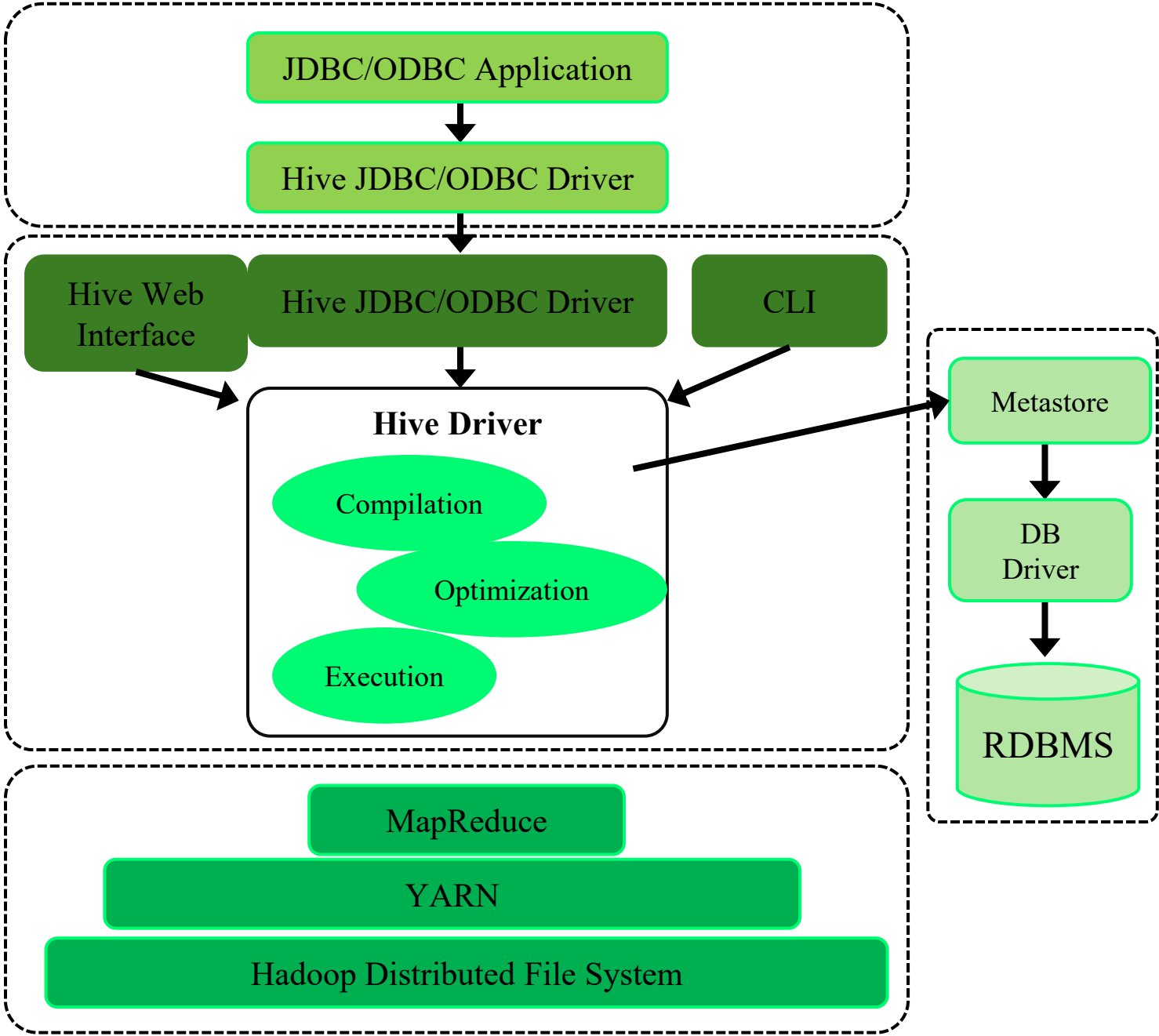
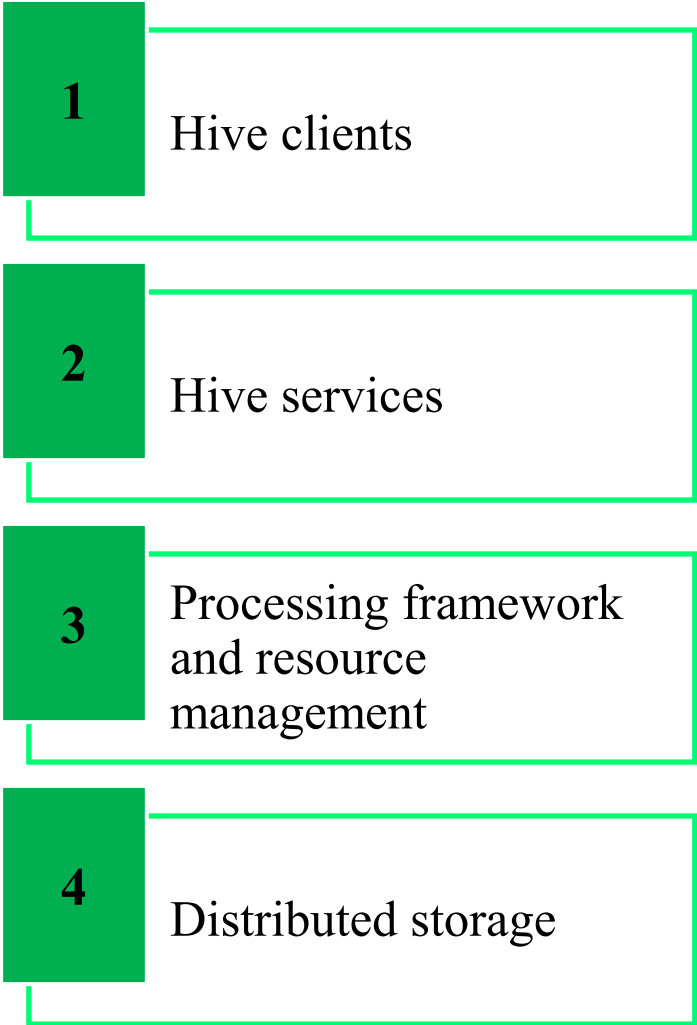
Executor:

- After compilation and optimization, the executor executes the tasks.
- It interacts with the job tracker of Hadoop to schedule tasks to be run.
- It takes care of pipelining the tasks by making sure that a task with dependency gets executed only if all other prerequisites are run.

CLI, UI, and Thrift Server:

- A command-line interface (CLI) provides a user interface for an external user to interact with Hive by submitting queries, instructions and monitoring the process status.
- Thrift server allows external clients to interact with Hive over a network, similar to the JDBC or ODBC protocols

2.2.1 Hive Components



Hive components:

Hive Clients: Hive supports applications that are written in many languages like Java, Python, C++, etc., using Thrift, JDBC, and ODBC drivers.

- a. Thrift Clients – As the Apache Hive server is based on Thrift, so it can serve the request from all those languages that support Thrift.
Apache Thrift is basically protocols which define how you made connections between clients and servers. Apache Hive uses Thrift to allow remote users to make a connection with HiveServer (The thrift server) to connect to it and submit queries.
- b. JDBC Clients – Apache Hive allows Java applications to connect to it using the JDBC driver. It is defined in the class `apache.hadoop.hive.jdbc.HiveDriver`.
- c. ODBC Clients – ODBC Driver allows applications that support ODBC protocol to connect to Hive.

Hive Services

Apache Hive provides various services to perform queries using HQL language like:

- a) CLI(Command Line Interface) – This is the default shell that Hive provides, in which you can execute your Hive queries and command directly.
- b) Web Interface – Hive also provides web based GUI for executing Hive queries and commands.
- c) Hive Server – It is built on Apache Thrift and thus is also called as Thrift server. It allows different clients to submit requests to Hive and retrieve the final result.
- d) Hive Driver – Driver is responsible for receiving the queries submitted Thrift, JDBC, ODBC, CLI, Web UL interface by a Hive client.

Processing Framework and Resource Management: Internally, Hive queries are executed using the Hadoop MapReduce framework.

Distributed Storage: Hive is installed on Hadoop and uses HDFS data.

Checkpoint

1. Point out the correct statement.
 - A) **Hive is not a relational database, but a query engine that supports the parts of SQL specific to querying data**
 - B) **Hive is a relational database with SQL support**
 - C) **Pig is a relational database with SQL support**
 - D) **All of the above**

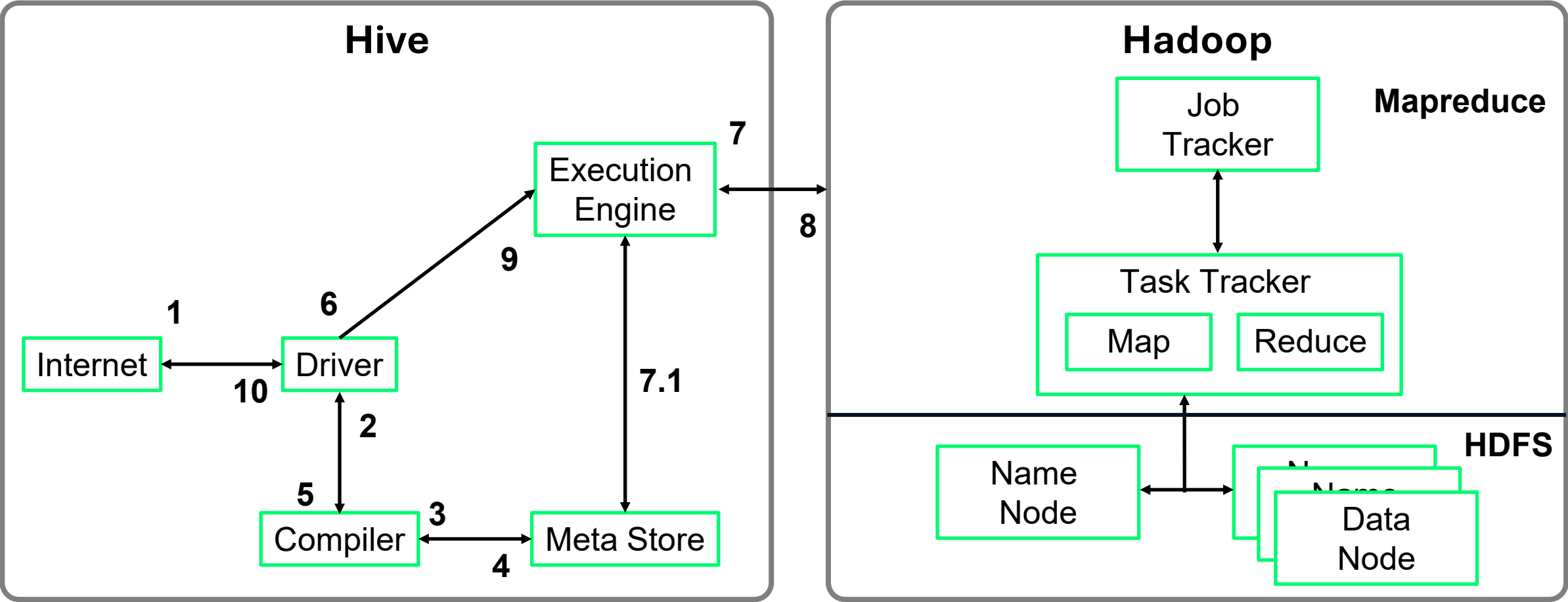
Answer:

1. a. Hive is not a relational database, but a query engine that supports the parts of SQL specific to querying data

Explanation: Hive is a SQL-based data warehouse system for Hadoop that facilitates data summarization, ad hoc queries, and the analysis of large datasets stored in Hadoop-compatible file systems.

2.3 Working of Hive

The following picture shows the working of Hive.



2.3 Working of Hive

1. Execute Query: The Hive interface, such as Command Line or Web UI sends query to the Driver (any database driver such as JDBC, ODBC, etc.) to execute.

2. Get Plan: The driver takes the help of query compiler that parses the query to check the syntax and query plan or the requirement of query.

3. Get Metadata: The compiler sends metadata request to Metastore (any database).

4. Send Metadata: Metastore sends metadata as a response to the compiler.

5. Send Plan: The compiler checks the requirement and resends the plan to the driver. Up to here, the parsing and compiling of a query is complete.

6. Execute Plan: The driver sends the execute plan to the execution engine.

7. Execute Job: Internally, the process of execution job is a MapReduce job. The execution engine sends the job to JobTracker, which is in Name node and it assigned this job to TaskTracker, which is in Data node. Here, the query executes MapReduce job.

7.1 Metadata Ops: Meanwhile in execution, the execution engine can execute metadata operations with Metastore.

8. Fetch Result: The execution engine receives the results from Data nodes.

9. Send Results: The execution engine sends those resultant values to the driver.

10. Send Results: The driver sends the results to Hive Interfaces.

2.4 HIVE Metastore

- ❑ Hive Metastore is the central repository for metadata.
- ❑ It stores metadata for Hive tables (like their schema and location) and partitions in a relational database
- ❑ Hive metastore consists of 2 fundamental units:
 - ❑ **Metastore Service** (the software interface handling requests) and
 - ❑ **Backend Database** (the relational database storing the actual metadata)

Metastore Service

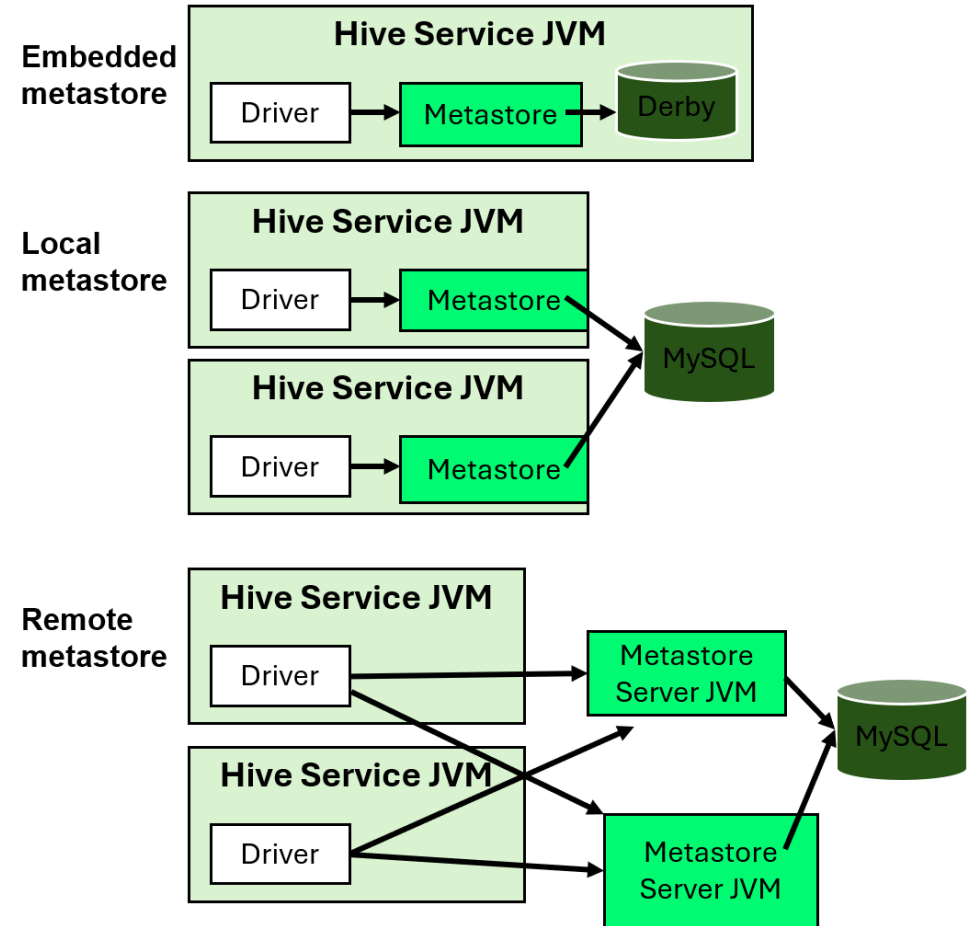
- Acts as the central server interface.
- Processes requests from tools like Spark, Presto, or Hive.
- Uses the Thrift protocol to communicate over the network.
- Manages client access to table schemas and locations.

Backend Database

- Operates as the underlying storage layer.
- Uses relational databases like MySQL or PostgreSQL.
- Saves table names, column types, and partition details.
- Stores file storage paths pointing to HDFS or cloud storage.

→ Modes of Metastore:

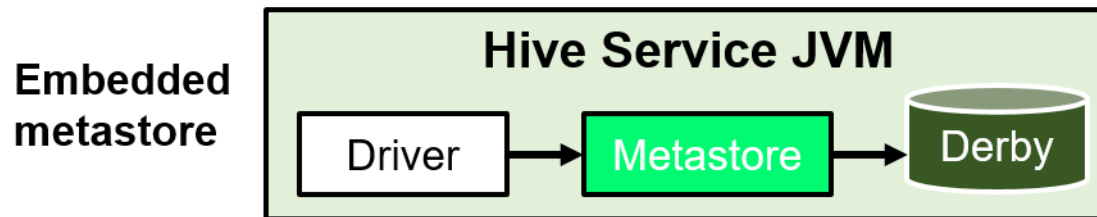
- ↳ Embedded
- ↳ Local
- ↳ Remote



Modes of Metastore:

- **Embedded:**

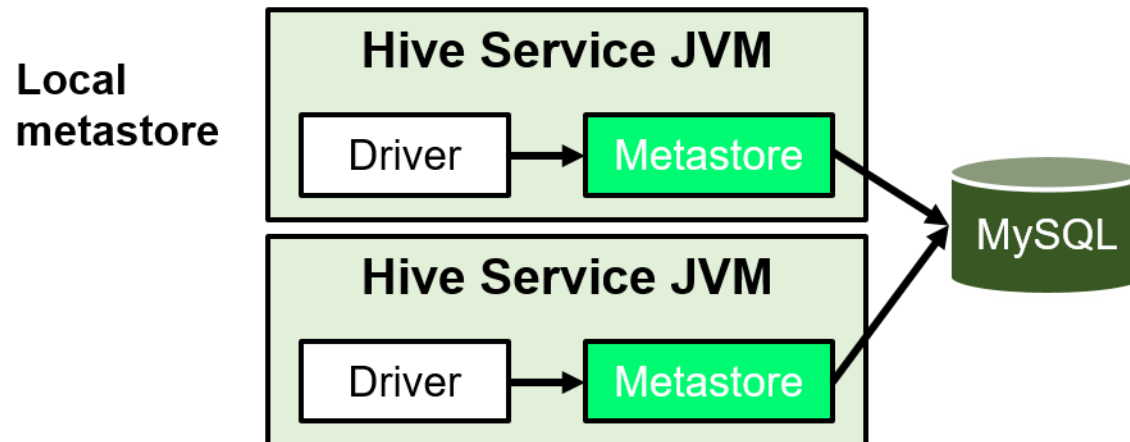
- In Hive by default, metastore service and hive services run in the same JVM.
- In this mode, Data in local file system are stored using the embedded derby database.
- It has the limitation that, as both metastore and hive service are in one JVM and having only one embedded Derby database to connect to disk, there is only one Hive session be active at a time.



Modes of Metastore:

• Local:

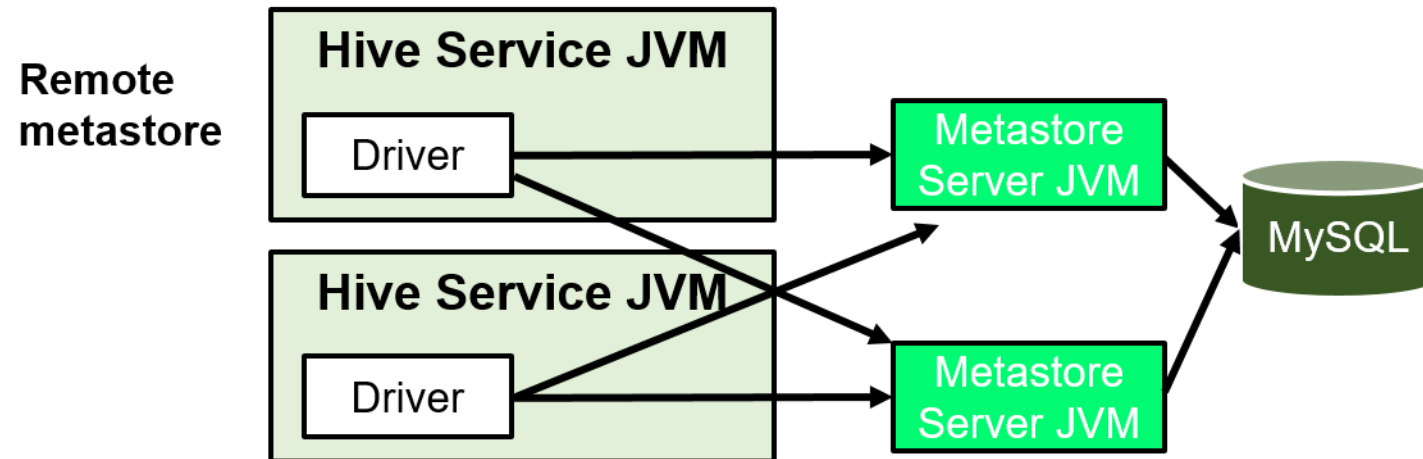
- Hive is a SQL based framework, that should have multiple sessions.
- In Local mode, multiple Hive sessions are allowed.
- This can be achieved by using any JDBC application like MySQL that runs in a separate JVM.
- In this process, metastore service runs in the same JVM as Hive service it is named as local metastore.
- The database is connected in a separate process, either on the same machine or on a remote machine



Modes of Metastore:

- **Remote:**

- In this mode, metastore and hive services run in a separate JVM.
- Thrift network APIs are used by different processes to communicate among them.
- Based on availability metastore services are scaled up.
- In this mode, the clients have no need to share database credentials with each Hive user to access the metastore database.



What did You Grasp?

1. Which option is not correct about metastore?
 - A) It stores the table location**
 - B) It has information about the number of partitions and number of buckets**
 - c) It can give you time at which table is created**
 - D) It stores the actual data**

Answer:

d. Metastore stores only metadata of table. The actual data are stored in hdfs.

2.5 Limitations of Hive

Below are a few limitations of HIVE:

- Traditional Hive (prior to version 0.14) did not support row-level inserts, updates, or deletes. Modern Hive fully supports row-level modifications using ACID-compliant tables.
- Hive is not designed to replace transactional RDBMS engines. However, modern versions (Hive 3.x+) provide full ACID compliance and transaction support with optimized background compaction to minimize performance impacts.
- Hive is strictly an OLAP (Online Analytical Processing) system designed for batch processing, not an OLTP system. Because it is optimized for high-throughput scans rather than real-time data access, query execution has inherently high latency.
- The SQL keywords NULL and null are treated identically in syntax. However, Hive distinguishes between a true database NULL (stored by default as \N in underlying text files) and literal text strings like "NULL" or "null"

2.6 Hive Compared with Traditional Databases

Feature	Apache Hive	Traditional RDBMS
Schema Management	Schema on Read (applies structure during query execution)	Schema on Write (enforces strict structure before saving)
Data Modifications	Optimized for Append (supports ACID transactions, but best for batch loads)	Frequent Read/Write (highly optimized for concurrent updates and deletes)
Data Volume Scale	Built for Petabyte+ Scale (handles massive big-data pools)	Typically handles Gigabytes to Terabytes (distributed RDBMS can scale higher)
Workload Type	OLAP (Online Analytical Processing for batch analytics)	OLTP (Online Transactional Processing for live applications)
Analysis Style	Historical batch analysis (scans massive datasets over minutes/hours)	Real-time dynamic analysis (fetches specific individual records in milliseconds)
Scalability	Highly Scalable (scales out horizontally using commodity servers)	Expensive to Scale (traditionally scales vertically by buying bigger hardware)

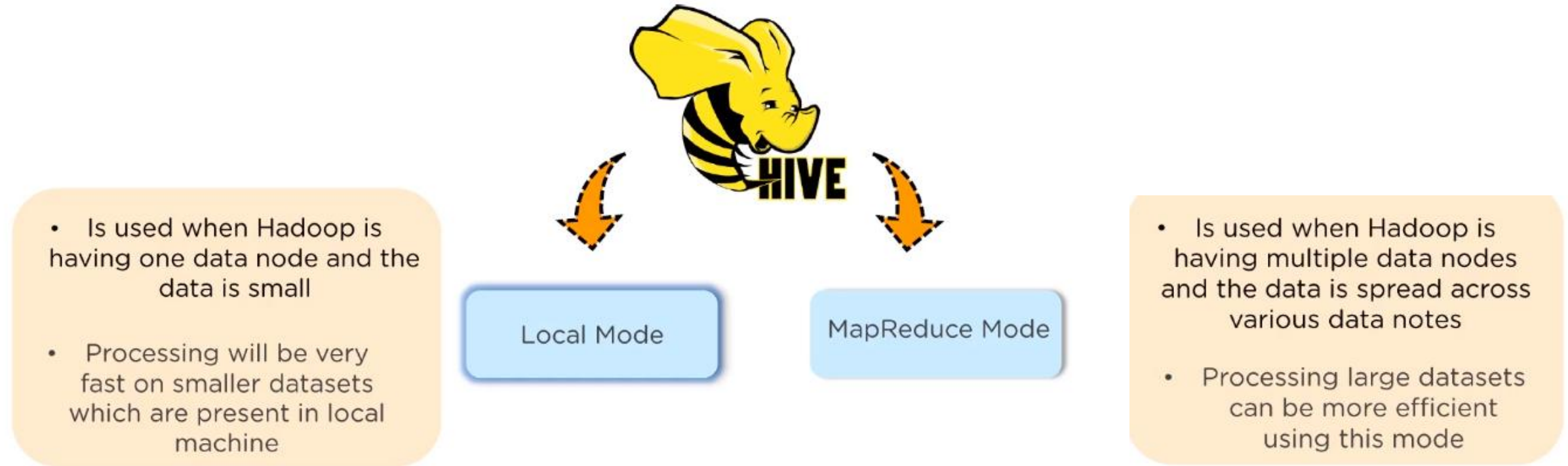
- In RDBMS, a table's schema is enforced at a data load time, if the data being loaded doesn't conform to the schema, then it is rejected. This design is called **schema on write**.
- But Hive doesn't verify the data when it is loaded, but rather when it is retrieved. This is called **schema on read**.
- Schema on read makes for a very fast initial load, since the data does not have to be read, parsed, and serialized to disk in the database's internal format.

What did You Grasp?

1. In which scenario, Hive is used?
 - A) Completely unstructured data
 - B) Structured data**
 - C) Any kind of data
 - D) None of the above

Different Modes of HIVE

Hive operates in 2 different modes depending on the number and size of the data nodes.



Features of Hive



Tables are used which are similar to RDBMS hence easier to understand

Using Hive QL, multiple users can simultaneously query data



Use of SQL like language called HiveQL which is easier than long codes

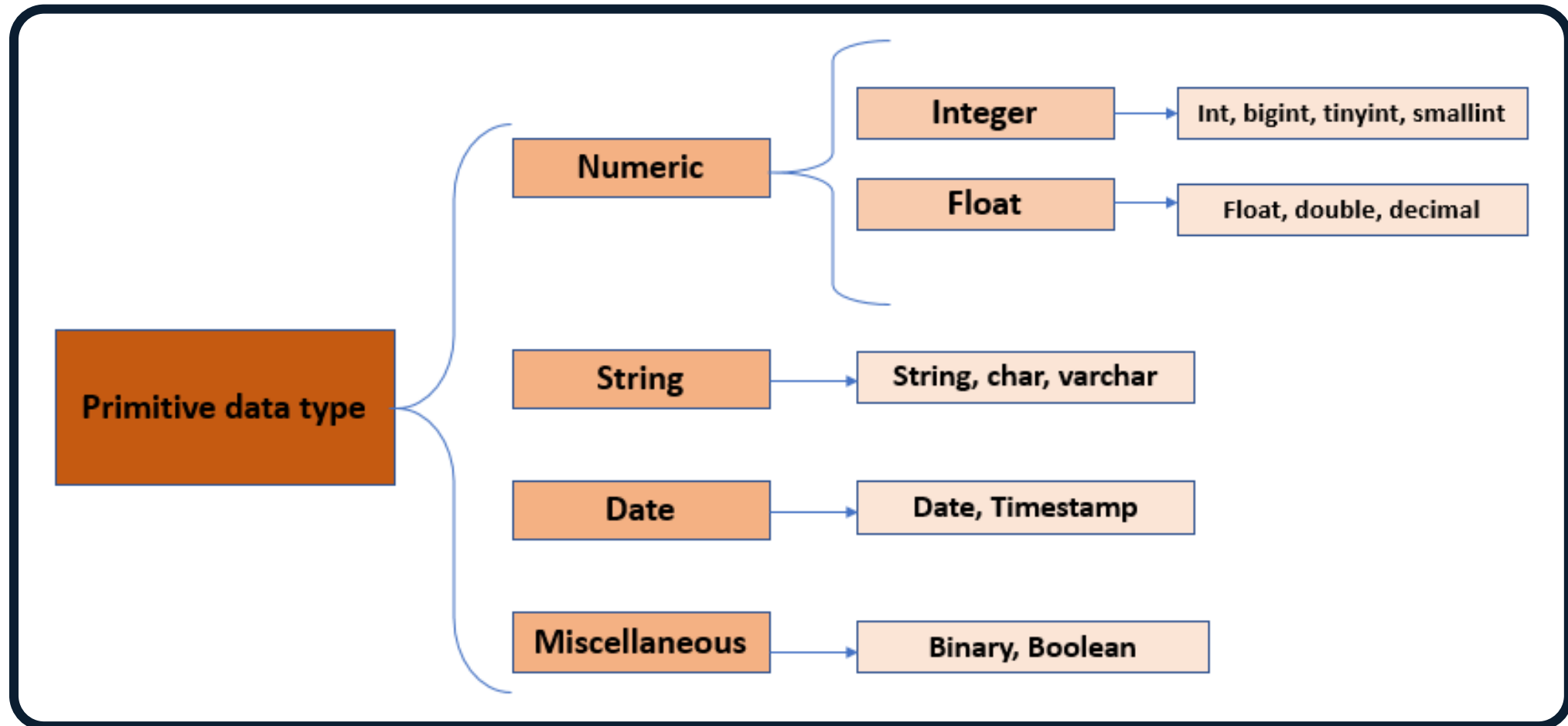


Hive supports variety of data formats



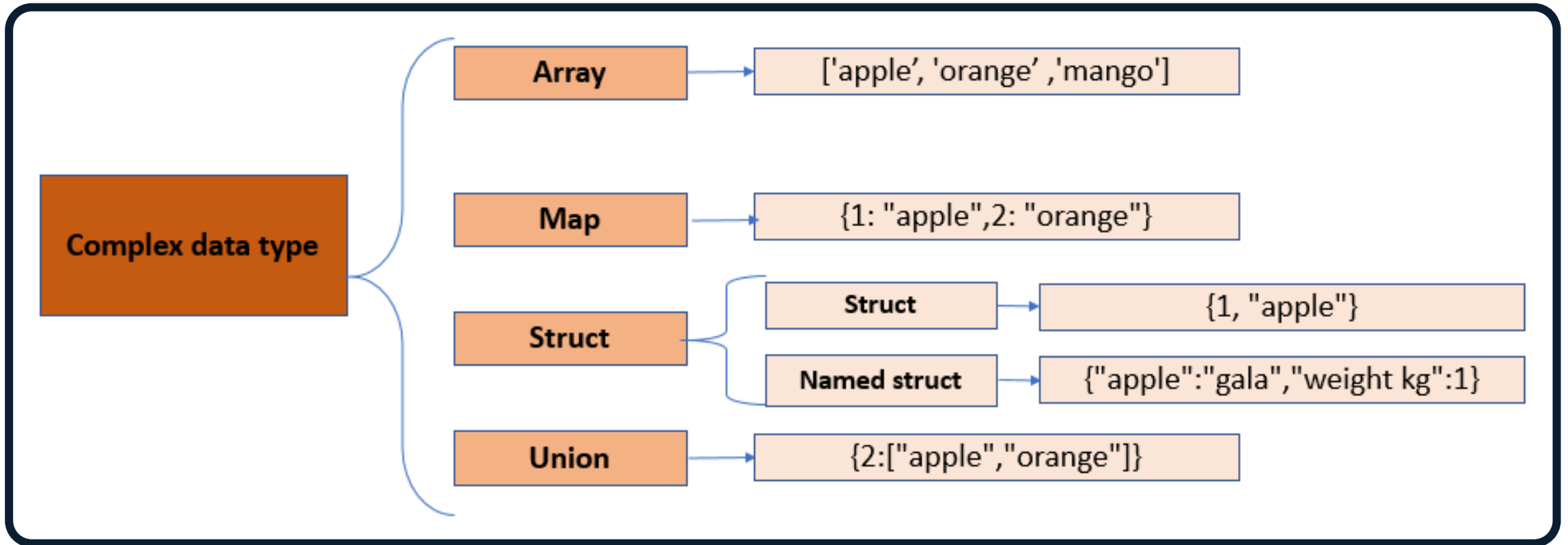
2.7 Hive Data Types - Primitive Data Type

The picture shows the different primitive data types in Hive.



2.7.1 Hive Data Type - Complex Data Type

The picture shows the different complex data types in Hive.



Examples

```
CREATE TABLE employee (  
  id INT,  
  name STRING,  
  salary DECIMAL(10,2),  
  is_active BOOLEAN,  
  join_date DATE,  
  contact STRUCT<email:STRING, phone:STRING>,  
  skills ARRAY<STRING>  
)
```

```
CREATE TABLE employees (  
  name STRING,  
  salary FLOAT,  
  subordinates ARRAY<STRING>,  
  deductions MAP<STRING, FLOAT>,  
  address STRUCT<street:STRING, city:STRING, state:STRING, zip:INT>);
```

2.7.2 Implicit Conversion Between Primitive Data Types

Implicit Conversion Between Primitive Data Types

The two ways of data type conversions

→ Primitive Type

↳ Number

- DOUBLE
 - FLOAT
 - BIGINT
 - INT
 - SMALLINT
 - TINYINT
- DECIMAL (Can be converted to String, varchar only)
- STRING (Can be converted to Varchar, Double, Decimal)
- VARCHAR (Can be converted to String, Double, Decimal)
 - DATE (Converted to String, varchar)
 - TIMESTAMP (same as date)

↳ BOOLEAN

↳ BINARY

Explicit Conversion

→ Cast functions returns null

In the above hierarchy, implicit conversion is allowed from child to an ancestor.
Eg: TINYINT can be converted to any other numeric data type whereas BIGINT can only be converted to FLOAT or DOUBLE, but the reverse, i.e., Binary and Boolean data types will not be converted to other data type implicitly.

What did You Grasp?

1. Integral literals are assumed to be _____ by default.
 - A) SMALL INT
 - B) INT**
 - C) BIG INT
 - D) TINY INT

Integral literals are assumed to be INT by default, unless the number exceeds the range of INT in which case it is interpreted as a BIGINT, or if one of the following postfixes is present on the number.

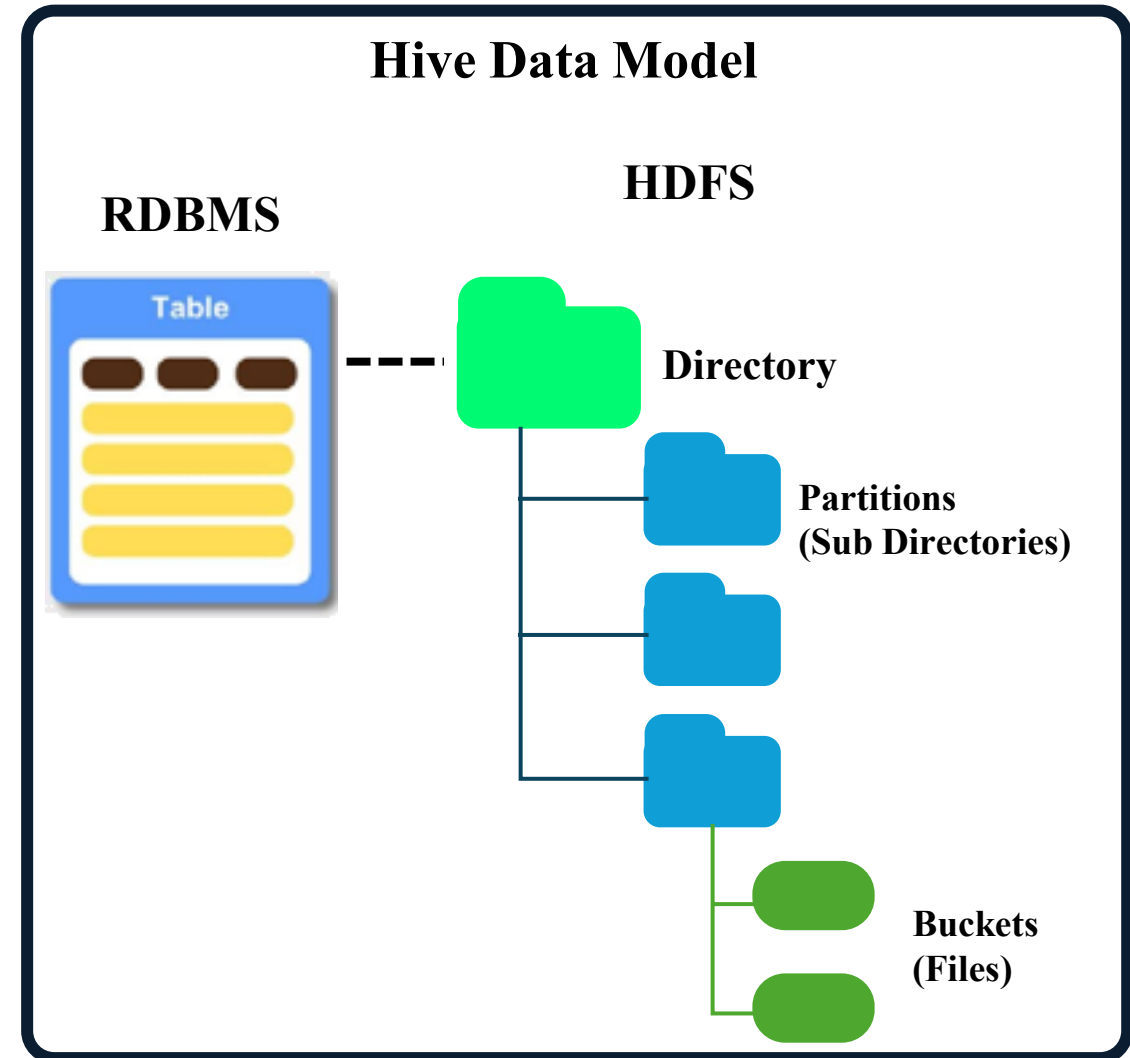
2.8 Hive Data Models

Hive Data Model

- Database/directory
- Tables
- Partitions
- Buckets

Hive Database

- “Default” database
- /user/hive/warehouse/<database name>.db
- Each database is a parent folder in HDFS



Points To Note

- A Database in Hive is nothing but HDFS Directory
- A Table in Hive is nothing but HDFS Directory
- A Partition in Hive is nothing but HDFS Directory

- Warehouse Directory is the base directory where directories related to databases, tables go by default.

- It is controlled by `hive.metastore.warehouse.dir`.

- You can get the value by saying `SET hive.metastore.warehouse.dir;`

The default value is typically `/user/hive/warehouse` in HDFS.

2.8.1 Creation of Sample Hive Database

```
CREATE DATABASE <DATABASE NAME>;  
CREATE DATABASE IF NOT EXISTS <DATABASE NAME>;
```

The keyword 'SCHEMA' can be used instead of 'DATABASE' while creating the database.

Also, add comments, location in which database should reside, other properties. Using below command.

```
CREATE (DATABASE|SCHEMA) [IF NOT EXISTS] DATABASE_NAME  
[COMMENT DATABASE_COMMENT]  
[LOCATION HDFS_PATH]  
[WITH DBPROPERTIES (PROPERTY_NAME=PROPERTY_VALUE, ...)]
```

- The CREATE DATABASE command creates the database under HDFS at the default location: /user/hive/warehouse.
- Hive creates a directory for each database. Tables in that database will be stored in sub directories of the database directory. The exception is tables in the default database, which doesn't have its own directory.

2.8.1 Creation of Sample Hive Database

The following screenshots show the commands for creating a database in Hive.

```
hive> create database test_database;  
OK  
Time taken: 0.038 seconds  
  
hive> create database if not exists test_database;  
OK  
Time taken: 0.012 seconds  
  
hive> create database if not exists test_database  
    > COMMENT 'hive test database'  
    > LOCATION '/apps/hive/warehouse'  
    > WITH DBPROPERTIES ('creator'='test', 'date'='2018-11-01');  
OK  
Time taken: 0.019 seconds  
hive> █
```

To check properties of database: describe database extended test_database

2.8.2 List Hive Database

Show created Hive database

```
hive> SHOW DATABASES LIKE 'test_datab.*';
OK
test_database
Time taken: 0.016 seconds, Fetched: 1 row(s)
hive> █
```

Syntax to see the databases that already exists in hive.

SHOW DATABASES;

SHOW DATABASES LIKE 'database_name.*';

Show described format of database

```
hive> DESCRIBE DATABASE test_database;
OK
+-----+-----+-----+
|db_name|  comment  |      location      |
+-----+-----+-----+
|test_database|hive test database|hdfs://localhost:8020/apps/hive/warehouse|
+-----+-----+-----+
1 row selected (0.352 seconds)
```

Note: Default location in Cloudera is /user/hive/warehouse/

2.8.3 Database Commands Usage

Use database

```
hive> use test_database;  
OK  
Time taken: 0.035 seconds  
hive> set hive.cli.print.current.db=true;  
hive (test_database)> █
```

Alter database

```
hive (test_database)> ALTER DATABASE test_database SET OWNER user tester;  
OK  
Time taken: 0.241 seconds  
hive (test_database)> █
```

Drop database

```
hive (test_database)> DROP DATABASE IF EXISTS test_database;  
OK  
Time taken: 0.31 seconds
```

```
hive (test_database)> show databases like 'test_dat.*';  
OK  
Time taken: 0.015 seconds  
hive (test_database)> █
```

Drop database with CASCADE

```
hive (test_database)> DROP DATABASE IF EXISTS test_database CASCADE;  
OK  
Time taken: 0.016 seconds  
hive (test_database)> █
```

CASCADE is used while dropping database which has tables created under it

Lab Activity

- Open terminal
- Type Hive to use hive shell
- Show databases
- Create database dbname
- Drop database dbname [cascade]
- Use dbname
- Open another terminal
- Create file employee.csv using gedit
- Check path using pwd
- Ensure data is comma separated
- Go back to hive shell
- Create employee table
- Describe table
- Load data from employee.csv file (See next slide)

```
[cloudera@quickstart Documents]$ cat Employee.csv
Id,Name,Dept,Yoj,Salary
1,Rose ,IT ,2012,26000
2,Sam,Sales,2012,22000
3,Mike,HR,2013,30000
4,Nick,SC ,2013,20000
[cloudera@quickstart Documents]$
```

```
hive> create table employee
> (Id INT, Name STRING, Dept STRING, Yoj INT, salary INT)
> row format delimited fields terminated by ','
> tblproperties ("skip.header.line.count"="1");
```

Load Data from Local inpath

```
hive> show tables;
OK
employee
Time taken: 0.056 seconds, Fetched: 1 row(s)
hive> describe employee;
OK
id                int
name              string
dept             string
yoj              int
salary           int
Time taken: 0.201 seconds, Fetched: 5 row(s)
hive> select * from employee;
OK
Time taken: 0.621 seconds
hive> LOAD DATA LOCAL INPATH
> '/home/cloudera/Documents/Employee.csv'
> INTO TABLE employee;
Loading data to table office.employee
Table office.employee stats: [numFiles=1, totalSize=116]
OK
Time taken: 1.147 seconds
```

```
hive> select * from employee;
OK
1      Rose      IT      2012      26000
2      Sam       Sales   2012      22000
3      Mike      HR      2013      30000
4      Nick      SC      2013      20000
Time taken: 0.09 seconds, Fetched: 4 row(s)
hive> select count(*) from employee;
Query ID = cloudera_20190328091919_b0a18254-a63d-437f-af94-eef78162217f
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1553695997344_0006, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1553695997344_0006/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1553695997344_0006
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2019-03-28 09:20:16,881 Stage-1 map = 0%, reduce = 0%
```

2.8.4 Hive Data Definition Language

Supported hive DDL functionalities are as follows:

- CREATE DATABASE/SCHEMA, TABLE, VIEW, FUNCTION, INDEX
- DROP DATABASE/SCHEMA, TABLE, VIEW, INDEX
- TRUNCATE TABLE
- ALTER DATABASE/SCHEMA, TABLE, VIEW
- SHOW DATABASES/SCHEMAS, TABLES, TBLPROPERTIES, VIEWS, PARTITIONS, FUNCTIONS, INDEX[ES], COLUMNS, CREATE TABLE
- DESCRIBE DATABASE/SCHEMA, table_name, view_name

2.8.5 Managed Table

Managed Table

Hive tables are created as internal or managed tables:

- When the managed table is created by default data stores in the warehouse directory.
- Data loss problem is there with internal table.
- Global usage is not there.

The process of creating a managed table is as follows:

- Dropping DDL, deletes data in hdfs location
- Default location “/user/hive/warehouse/”
- Created under given database location

2.8.6 External Table

External Table

External table is a table for which the storage is not managed by Hive.

The data loss problem is not present in External tables. The process of creating an external table is as follows:

- The table can be created on existing hdfs file
- Specify LOCATION in DDL
- Table when dropped metadata alone gets deleted. Data remains in hdfs

2.8.7 Syntax for Creation of Table in Hive

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS] [database_name.]table_name
(
    column_name data_type [COMMENT 'column_comment'],
    ...
)
[COMMENT 'table_comment']
[PARTITIONED BY
    (partition_column data_type [COMMENT 'comment'], ...)]
[CLUSTERED BY
    (column_name, ...)
    [SORTED BY (column_name [ASC|DESC], ...)]
    INTO num_buckets BUCKETS]
[SKEWED BY (column_name, ...)
    ON ((column_value, ...), ...)
    [STORED AS DIRECTORIES]]
[
    [ROW FORMAT row_format]
    [STORED AS file_format
    | STORED BY 'storage.handler.class.name'
    [WITH SERDEPROPERTIES (...)]]
]
[LOCATION 'hdfs_path']
[TBLPROPERTIES (property_name='property_value', ...)]
[AS select_statement];
```

In the Syntax, key words are to be noted, such as:

TEMPORARY – Used when user needs to create a table that is temporary for that session

EXTERNAL – Used when data were already present in HDFS and need a schema

IF NOT EXISTS – To suppress the warning, if table already exists

COMMENT – This can be provided both at column level and table level

PARTITIONED BY – Used when the table needs to be partitioned by one or more columns

CLUSTERED BY along with num_buckets BUCKETS – Used when bucketing is used

SORTED BY – Is part of bucketing concept

SKEWED BY – Used based on query optimization

ROW FORMAT – User can specify different row formats available like text, ORC, Avro, etc.

STORED AS – Storage format of data

LOCATION – This needs to be specified while creating EXTERNAL tables

TBLPROPERTIES – To add any properties that define the creator or a table or group

2.8.8 Creation of Sample Managed Table in Hive

The screenshots show the commands for creating a managed table in Hive.

```
hive (test_database)> CREATE TABLE managed_table (dummy STRING);
OK
Time taken: 0.194 seconds
hive (test_database)> show create table managed_table;
OK
CREATE TABLE `managed_table` (
  `dummy` string)
ROW FORMAT SERDE
  'org.apache.hadoop.hive ql.io.orc.OrcSerde'
STORED AS INPUTFORMAT
  'org.apache.hadoop.hive ql.io.orc.OrcInputFormat'
OUTPUTFORMAT
  'org.apache.hadoop.hive ql.io.orc.OrcOutputFormat'
LOCATION
  'hdfs://localhost:8020/apps/hive/warehouse/test_database.db/managed_table'
TBLPROPERTIES (
  'COLUMN_STATS_ACCURATE'='{\"BASIC_STATS\": \"true\"}',
  'numFiles'='0',
  'numRows'='0',
  'rawDataSize'='0',
  'totalSize'='0',
  'transient_lastDdlTime'='1542554392')
Time taken: 0.118 seconds, Fetched: 17 row(s)
```

2.8.9 Creation of Sample External Table in Hive

The screenshots show the process of creating an external table in Hive.

```
hive (test_database)> CREATE EXTERNAL TABLE external_table          (dummy STRING)
                        > LOCATION '/apps/hive/warehouse/external_table';
OK
Time taken: 0.05 seconds
hive (test_database)> show create table external_table;
OK
CREATE EXTERNAL TABLE `external_table` (
  `dummy` string)
ROW FORMAT SERDE
  'org.apache.hadoop.hive ql.io.orc.OrcSerde'
STORED AS INPUTFORMAT
  'org.apache.hadoop.hive ql.io.orc.OrcInputFormat'
OUTPUTFORMAT
  'org.apache.hadoop.hive ql.io.orc.OrcOutputFormat'
LOCATION
  'hdfs://localhost:8020/apps/hive/warehouse/external_table'
TBLPROPERTIES (
  'numFiles'='0',
  'totalSize'='0',
  'transient_lastDdlTime'='1542554946')
Time taken: 0.076 seconds, Fetched: 14 row(s)
```

2.8.10 Creating a Sample Table in Hive Using CTAS

→ It is a copy of other table

The screenshot shows the process of creating a sample table using CTAS.

```
hive (test_database)> create table sample_ctas as SELECT * FROM managed_table;  
OK  
Time taken: 14.587 seconds  
hive (test_database)>
```

CTAS has the following restrictions:

- The table created cannot be a partitioned table
- The table created cannot be an external table
- The table created cannot be a list bucketing table
- A CTAS statement will trigger a map job for populating the data; even SELECT * itself does not trigger any MapReduce job.

Hive tables can be created as EXTERNAL or INTERNAL. This is a choice that affects how data is loaded, controlled and managed.

Use EXTERNAL tables when:

- The data is also used outside of Hive. For example, the data files are read and processed by an existing program that doesn't lock the files.
- Data needs to remain in the underlying location even after a DROP TABLE. This can apply if you are pointing multiple schemas (tables or views) at a single data set or if you are iterating through various possible schemas.
- You want to use a custom location, such as ASV.
- Hive should not own data and control settings, dirs, etc., you have another program or process that will do those things.
- You are not creating table based on existing tables (AS SELECT).

Use INTERNAL tables when:

- The data is temporary.
- You want Hive to completely manage the lifecycle of the table and data.

Data Loading & Insertion

-- Load data into table from HDFS

```
LOAD DATA INPATH '/user/hadoop/employee.csv' INTO TABLE employee;
```

-- Load from local file system

```
LOAD DATA LOCAL INPATH '/home/cloudera/Documents/employee.csv' INTO TABLE  
employee;
```

-- Insert single row

```
INSERT INTO TABLE employee VALUES (1, 'Deepak', 30000);
```

-- Insert data from another table

```
INSERT INTO TABLE employee SELECT * FROM employee_temp;
```

-- Insert overwrite (replace data)

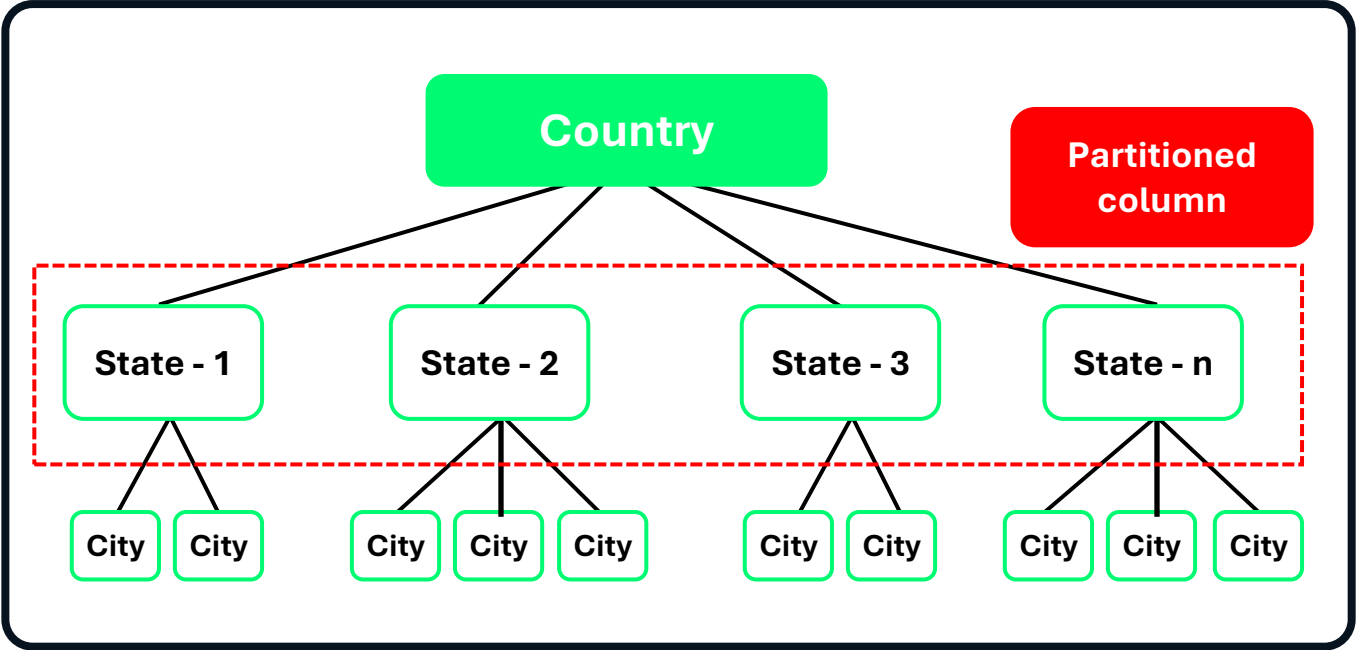
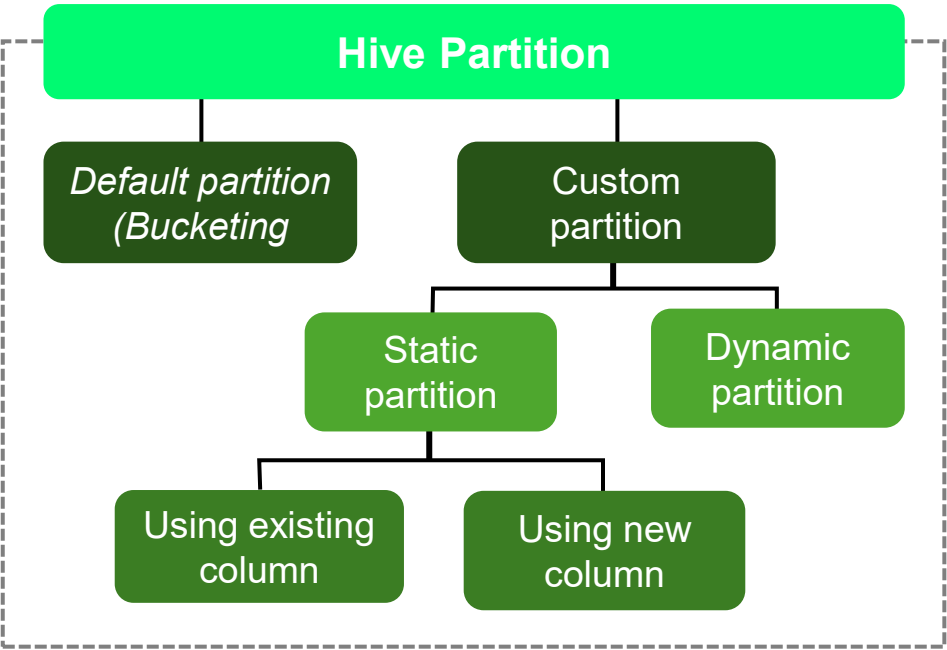
```
INSERT OVERWRITE TABLE employee SELECT * FROM employee_temp;
```

2.9 Partitions

- It is a method of separating a table into related parts based on the column values like date, city and department.
- Partitioned data stored in different subdirectory
- It improves query performance.
- It is very easy to do queries on partition table.

2.9. Partitions

The figures show partitions in Hive and sample partitions.



Why we use Partition?

- By default, a simple query in Hive scans the whole Hive table.
- This slows down the performance when querying a large-size table.
- The issue could be resolved by creating Hive partitions, which is very similar to what's in the RDBMS.
- In Hive, each partition corresponds to a predefined partition column(s) and stores it as a subdirectory in the table's directory in HDFS.
- When the table gets queried, only the required partitions (directory) of data in the table are queried, so the I/O and the time of the query is greatly reduced.

Hive Static Partitioning

- Insert input data files individually into a partition table is Static Partition.
- You “statically” add a partition in the table and move the file into the partition of the table.
- We can alter the partition in the static partition.
- You can get the partition column value from the filename, day of date etc without reading the whole big file.
- If you want to use the Static partition in the hive you should set property
set hive.mapred.mode = strict
This property set by default in hive-site.xml.

2.9.2 Dynamic Partitions

Benefits

Dynamic partition benefits:

- Columns whose values are only known at EXECUTION TIME.
- We use dynamic partition while loading from an existing table that is not partitioned.
- We use dynamic partition while unknown values for partition columns.
- Usually dynamic partition load the data from non partitioned table.
- Dynamic partition takes more time in loading data compared to static partition.
- We can't perform alter on dynamic partition. # Drop partition is allowed
- Perform dynamic partition on the hive external table and managed table.

```
set hive.exec.dynamic.partition=true  
set hive.exec.dynamic.partition.mode=nonstrict
```

2.9.1 Create Partitions

The following screenshot shows the command for creating partitioned table.

```
Hive> CREATE TABLE employee_partitioned
>(
> name string,
> work_place ARRAY<string>,
> gender_age STRUCT<gender:string,age:int>,
> skills_score MAP<string,int>,
> depart_title MAP<STRING,ARRAY<STRING>>
>)
>PARTITIONED BY (Year INT, Month INT)
>ROW FORMAT DELIMITED
>FIELDS TERMINATED BY '|'
>COLLECTION ITEMS TERMINATED BY ','
>MAP KEYS TERMINATED BY ':';
No rows affected (0.34 seconds)
```

2.9.3 View Partitions

The screenshots show the creation of partitions and how they look after loading data.

```
hive (test_database)> insert into city_partitioned partition (Ctry_code='IND',st_code='AP') values
('Hyderabad',100,'AndhraPradesh','India');
Query ID = id_20181118103417_abc123
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2018-11-18 10:34:25,699 Stage-1 map = 0%, reduce = 0%
2018-11-18 10:34:32,980 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.99 sec
MapReduce Total cumulative CPU time: 2 seconds 990 msec
Loading data to table test_database.city_partitioned partition (ctry_code=IND, st_code=AP)
Partition test_database.city_partitioned{ctry_code=IND, st_code=AP} stats: [numFiles=1, numRows=1,
totalSize=629, rawDataSize=283]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.99 sec HDFS Read: 5075 HDFS Write: 741 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 990 msec
OK
Time taken: 16.964 seconds
hive (test_database)> dfs -ls /apps/hive/warehouse/test_database.db/city_partitioned/ctry_code=IND;
Found 1 items
drwxrwxrwt - test hdfs      0 2018-11-18 10:34
/apps/hive/warehouse/test_database.db/city_partitioned/ctry_code=IND/st_code=AP
hive (test_database)>
```

2.9.4 Drop Partitions

The screenshots show how partitions are dropped.

```
hive (test_database)> ALTER TABLE city_partitioned DROP IF EXISTS PARTITION (ctry_code='IND', st_code='AP');
Dropped the partition ctry_code=IND/st_code=AP
OK
Time taken: 0.587 seconds
hive (test_database)> █
hive (test_database)> dfs -ls /apps/hive/warehouse/test_database.db/city_partitioned;
hive (test_database)> █
```

Partitioning: Example

CREATE TABLE table_name (column1 data_type, column2 data_type) PARTITIONED BY (partition1 data_type, partition2 data_type,);

File1:

tab1/clientdata/file1

id, name, dept, yoj

1, sunny, SC, 2009

2, animesh, HR, 2009

3, sumeer, SC, 2010

4, sarthak, TP, 2010

Partitioning: Example

Now, let us partition above data into two files using years

tab1/clientdata/2009/file2

1, sunny, SC, 2009

2, animesh, HR, 2009

tab1/clientdata/2010/file3

3, sumeer, SC, 2010

4, sarthak, TP, 2010

Now when we are retrieving the data from the table, only the data of the specified partition will be queried. Creating a partitioned table is as follows:

```
CREATE TABLE table_tab1 (id INT, name STRING, dept STRING, yoj INT) PARTITIONED BY (year STRING);
```

```
LOAD DATA LOCAL INPATH 'tab1/clientdata/2009/file2' OVERWRITE INTO TABLE studentTab PARTITION (year='2009');
```

```
LOAD DATA LOCAL INPATH 'tab1/clientdata/2010/file3' OVERWRITE INTO TABLE studentTab PARTITION (year='2010');
```

Dynamic Partitioning Example:

Enable the dynamic partition by using the following commands: -

```
hive> set hive.exec.dynamic.partition=true;  
hive> set hive.exec.dynamic.partition.mode=nonstrict;
```

Create a dummy table to store the data.

```
hive> create table stud_demo(id int, name string, age int, institute string, course string)  
row format delimited  
fields terminated by ',';
```

Now, load the data into the table.

```
hive> load data local inpath '/home/apps/hive/student_details' into table stud_demo;
```

Dynamic Partition Example

Create a partition table by using the following command: -

```
hive> create table student_part (id int, name string, age int, institute string)
partitioned by (course string)
row format delimited
fields terminated by ',';
```

Now, insert the data of dummy table into the partition table.

```
hive> insert into student_part
partition(course)
select id, name, age, institute, course from stud_demo;
```

**#partitioned column should be
mentioned at the end**

```
hive> SHOW PARTITIONS student_part
```

```
hive> select * from student_part where course= "hadoop"; // query on partitioned data
```

Example 2: Steps for Dynamic Partitioning in Hive

1. Prepare source CSV file.
2. Load file into HDFS.
3. Create a staging table (dummy table).
4. Create a partitioned table (empty).
5. Enable dynamic partitioning mode.
6. Use `INSERT INTO ... PARTITION (col) with SELECT` query.
7. Verify with `SHOW PARTITIONS`.

Example 2: Dynamic Partitioning

Step 1: Create a Source File

Let's say we have a CSV file `student_data.csv` with this data:

```
1,Sunny,SC,2009
2,Animesh,HR,2009
3,Sumeer,SC,2010
4,Sarthak,TP,2010
5,Rohan,HR,2011
```

Columns:

id, name, dept, year

Step 2: Put File into HDFS

Upload the file into HDFS (Hive reads from HDFS):

```
hdfs dfs -mkdir /user/hive/student_src
hdfs dfs -put student_data.csv /user/hive/student_src/
```

Example 2: Dynamic Partitioning

Step 3: Create a Staging (Source/dummy) Table

This table just reads the raw file (non-partitioned).

```
CREATE TABLE student_src ( id INT, name STRING, dept STRING, year STRING)ROW FORMAT  
DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION '/user/hive/student_src';
```

```
SELECT * FROM student_src;
```

Step 4: Create a Partitioned Target Table

We will partition this table by year (but not insert anything yet).

```
CREATE TABLE student_part ( id INT, name STRING, dept STRING)  
PARTITIONED BY (year STRING)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

Step 5: Enable Dynamic Partitioning

Hive requires enabling dynamic partition mode:

```
SET hive.exec.dynamic.partition=true;
```

```
SET hive.exec.dynamic.partition.mode=nonstrict;
```

Step 6: Insert Data Using Dynamic Partition

Now we insert into student_part by telling Hive to automatically create partitions based on the year column values:

```
INSERT INTO TABLE student_part PARTITION (year) SELECT id, name, dept, year  
FROM student_src;
```

Hive will create partitions for each unique year found in the source file (2009, 2010, 2011).

```
SHOW PARTITIONS student_part; //Also view the files/directories created at hdfs locations
```

What did You Grasp?

1. The partitioning of a table in Hive creates more _____.
 - A) Subdirectories under the database name
 - B) Subdirectories under the table name**
 - C) Files under database name
 - D) Files under the table name
2. How do you check if a particular partition exists?
 - A) SHOW TABLE table_name.
 - B) SHOW PARTITIONS table_name**
 - C) DESCRIBE table_name.
 - D) SHOW TABLES.

Partitions Recap

- It is a method of separating a table into related parts based on the column values like date, city and department.
- Partitioned data stored in different subdirectory
- It improves query performance.
- It is very easy to do queries on partition table.

Hive Partitioning – Advantages and Disadvantages

Hive Partitioning Advantages

- Partitioning in Hive distributes execution load horizontally.
- In partition faster execution of queries with the low volume of data takes place. For example, search population from Gujarat returns very fast instead of searching entire world population.

Hive Partitioning Disadvantages

- There is the possibility of too many small partition creations- too many directories.
- Partition is effective for low volume data. But there some queries like group by on high volume of data take a long time to execute.

Issues in Partitioning

Partitioning gives effective results when,

- there are limited number of partitions
- comparatively equal sized partitions
- Not possible in all scenarios, i.e., when partitioning tables based on geographic locations like country,
 - some bigger countries will have large partitions (ex: 4-5 countries itself contributing 70-80% of total data)
 - where as small countries data will create small partitions (remaining all countries in the world may contribute to just 20-30 % of total data).

So, in these cases Partitioning will not be ideal.

To overcome the problem of over partitioning, Hive provides **Bucketing concept**, another technique for decomposing table data sets into more manageable parts.

2.10 Buckets

Hive Buckets

Based on Hash function

Uses CLUSTERED BY clause

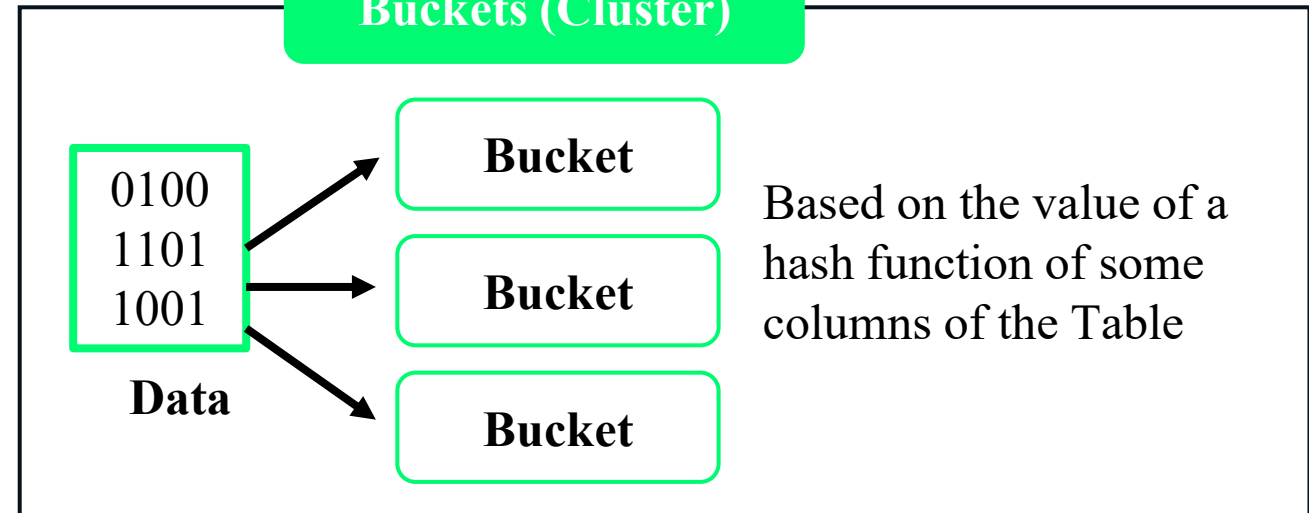
With or without partitioning on table

Equally distributed data files parts.

“SET hive.enforce.bucketing=true;”

$[\text{Hash}(\text{column(s)})] \text{ MOD } [\text{Number of buckets}]$

Buckets (Cluster)



The key details of Buckets:

- Hive partition separate table into the number of partitions and these partitions can be further subdivided into more manageable parts known as Buckets.
- It speeds up joins and sampling of data.
- Bucketing concept is based on Hash function.
- Buckets in hive is used in segregating of hive table-data into multiple files or directories. It is used for efficient querying.
- The data, i.e., present in that partition can be divided further into Buckets.
- The division is performed based on Hash of particular columns that we selected in the table.
- Buckets use some form of Hashing algorithm at back end to read each record and place it into buckets
- In Hive, we have to enable buckets by using the
set hive.enforce.bucketing=true;

2.10.2 Create Table with Bucket

The screenshot shows the process of creating a table with a bucket.

```
hive> create table sample_bucket
> (
> id int,
> name string,
> state string,
> zip string)
> clustered by (id) into 50 buckets
> row format delimited fields terminated by ',';
OK
Time taken: 0.94 seconds
```

2.10.3 Advantages of Bucketing

Sampling: When we want to test a table which has a huge amount of data or when we want to draw some patterns or when we want some aggregations (where accuracy is not our top priority) then we need to sample, i.e., we need to run the query on a smaller set of data of the entire table data with evenly distributed sample. There are two types of sampling:

1. Bucket Sampling : e.g. `SELECT * FROM T_USER_LOG_BUCKET TABLESAMPLE (BUCKET 1 OUT OF 4 AT USER_ID)`. It will select the data from the first buckets of each partition from T_USER_LOG_BUCKET table, otherwise in normal select we can't select data in such distributed and evenly manner.

2. Block Sampling: e.g. `SELECT * FROM T_USER_LOG_BUCKET TABLESAMPLE (20 PERCENT);`

2.10.4 View Data Buckets after Loading Table

The screenshots show how we can view the buckets after loading a table.

```
hive> dfs -ls /apps/hive/warehouse/sample_bucket;
Found items
-rwxrwxrwt 3 USERhdfs 0 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000000_0
-rwxrwxrwt 3 USERhdfs 598 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000001_0
-rwxrwxrwt 3 USERhdfs 596 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000002_0
-rwxrwxrwt 3 USERhdfs 601 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000003_0
-rwxrwxrwt 3 USERhdfs 601 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000004_0
-rwxrwxrwt 3 USERhdfs 596 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000005_0
-rwxrwxrwt 3 USERhdfs 596 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000006_0
-rwxrwxrwt 3 USERhdfs 601 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000007_0
-rwxrwxrwt 3 USERhdfs 601 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000008_0
-rwxrwxrwt 3 USERhdfs 601 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000009_0
-rwxrwxrwt 3 USERhdfs 596 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000010_0
-rwxrwxrwt 3 USERhdfs 617 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000011_0
-rwxrwxrwt 3 USERhdfs 657 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000012_0
-rwxrwxrwt 3 USERhdfs 654 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000013_0
-rwxrwxrwt 3 USERhdfs 669 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000014_0
-rwxrwxrwt 3 USERhdfs 658 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000015_0
-rwxrwxrwt 3 USERhdfs 646 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000016_0
-rwxrwxrwt 3 USERhdfs 679 2018-11-22 11:18 /apps/hive/warehouse/sample_bucket/000017_0
```

```
hive> dfs -cat /apps/hive/warehouse/sample_bucket/000001_0;
101,Sasi,Karnataka,560045
hive> dfs -cat /apps/hive/warehouse/sample_bucket/000004_0;
104,Kiran,Karnataka,560046
hive> dfs -cat /apps/hive/warehouse/sample_bucket/000014_0;
114,Rizvana,Karnataka,560024
214,Smitha,Karnataka,560046
hive> █
```

Bucketing

Example 1

- **Create table for storing data in buckets**

```
hive>Create table emp (id int, dept string, sal int)
clustered by (id) into 5 buckets
row format delimited fields terminated by ',';
```

```
set hive.enforce.bucketing = true;
```

- **Insert data into table**

```
Hive>Insert into emp values (102,'HR',25000);
.....
.....
```

- **Observe buckets and data at HDFS location:**
 - Hdfs dfs -ls /user/hive/warehouse/bigdata.db/emp

Bucketing

Example 2

- **Create a dummy table to store the data.**

```
hive>create table emp_demo (Id int, dept string , Salary float)  
row format delimited fields terminated by ',' ;
```

- **Now, load the data into the table.**

```
hive>load data local inpath '/home/cloudera/Documents/emp.csv' overwrite into table emp_demo;
```

- **Enable the bucketing by using the following command: -**

```
hive> set hive.enforce.bucketing = true;
```

Example 2 Continue

- **Create a bucketing table by using the following command: -**

```
hive>create table emp_bucket(Id int, dept string , Salary float)
clustered by (Id) into 5 buckets
row format delimited  fields terminated by ',' ;
```

- **Now, insert the data of dummy table into the bucketed table.**

```
hive> insert overwrite table emp_bucket select * from emp_demo;
```

- **Observe buckets and data at HDFS location:**

```
hdfs dfs -ls /user/hive/warehouse/bigdata.db/emp_bucket
```

Bucketing

Example 3 (Using Dynamic Partitioning)

Enable the bucketing and the dynamic partition by using the following commands: -

```
hive> set hive.enforce.bucketing = true;  
hive> set hive.exec.dynamic.partition=true;  
hive> set hive.exec.dynamic.partition.mode=nonstrict;
```

- **Create a dummy table to store the data.**

```
hive>create table emp_demo (Id int, dept string , salary float)  
row format delimited fields terminated by ',';
```

- **Now, load the data into the table.**

```
hive>load data local inpath '/home/cloudera/Documents/emp.csv' into table emp_demo;
```

Example 3 Continue

Create a partition table with buckets by using the following command: -

```
hive>create table emp_part_bucket(Id int, Salary int)
Partitioned by (dept string)
clustered by (Id) into 5 buckets
row format delimited  fields terminated by ',' ;
```

Now, insert the data of dummy table into the partition table.

```
hive> insert into emp_part_demo partition(dept)
select id,salary,dept from stud_demo;           #partitioning column should be last column
```

Hive>Show partitions emp_part_demo;

Check for buckets created under various partitions at HDFS location

What did You Grasp?

1. How can you enable buckets?
 - A) hive.bucketing.pruning
 - B) hive.enforce.bucketing**
 - C) hive.bucketing.location
 - D) hive.exec.bucketing

Assignment

Apply Bucketing on book_info table clustered by bookid into 4 partitions.

- a) Without partitioning
- b) With partitioning (Year)

Operators in HIVE

There are four types of operators in Hive:

- Relational Operators
- Arithmetic Operators
- Logical Operators
- Complex Operators

Relational Operators

- Relational Operators are used to compare the operands.
- Relational Operators generate a TRUE or FALSE value depending on condition.

Relational Operator	Operand types	Description
A = B	all primitive types	TRUE if expression A is equivalent to expression B otherwise FALSE
A != B	all primitive types	TRUE if expression A is not equivalent to expression B otherwise FALSE
A < B	all primitive types	TRUE if expression A is less than expression B otherwise FALSE
A <= B	all primitive types	TRUE if expression A is less than or equal to expression B otherwise FALSE
A > B	all primitive types	TRUE if expression A is greater than expression B otherwise FALSE
A >= B	all primitive types	TRUE if expression A is greater than or equal to expression B otherwise FALSE
A IS NULL	all types	TRUE if expression A evaluates to NULL otherwise FALSE
A IS NOT NULL	all types	FALSE if expression A evaluates to NULL otherwise TRUE
A LIKE B	strings	TRUE if string A matches the SQL simple regular expression B, otherwise FALSE.
A RLIKE B	strings	NULL if A or B is NULL, TRUE if any (possibly empty) substring of A matches the Java regular expression B, otherwise FALSE.
A REGEXP B	strings	NULL if A or B is NULL, TRUE if any (possibly empty) substring of A matches the Java regular expression B, otherwise FALSE.

Relational Operators Example

```
hive> SELECT * FROM employee WHERE Id=1241;
```

```
Hive> SELECT * from EMP WHERE department IS NOT NULL;
```

Arithmetic Operators

These operators support various common arithmetic operations on the operands.

- All of them return number types.

Operators	Operand	Description
$A + B$	all number types	Gives the result of adding A and B.
$A - B$	all number types	Gives the result of subtracting B from A.
$A * B$	all number types	Gives the result of multiplying A and B.
A / B	all number types	Gives the result of dividing B from A.
$A \% B$	all number types	Gives the reminder resulting from dividing A by B.
$A \& B$	all number types	Gives the result of bitwise AND of A and B.
$A B$	all number types	Gives the result of bitwise OR of A and B.
$A \wedge B$	all number types	Gives the result of bitwise XOR of A and B.
$\sim A$	all number types	Gives the result of bitwise NOT of A.

Arithmetic Operators Example

Hive> Select 20+50 from emp limit 1;

O/P:

70

Hive> SELECT id, dept, sal+1000 FROM emp;

???

This will display all rows of emp table and will add 1000 to each employee's salary.

Will the changes be reflected in the database?

Logical Operators

- The operators are logical expressions.
- All of them return either TRUE or FALSE

Operators	Operands	Description
A AND B	boolean	TRUE if both A and B are TRUE, otherwise FALSE.
A && B	boolean	Same as A AND B.
A OR B	boolean	TRUE if either A or B or both are TRUE, otherwise FALSE.
A B	boolean	Same as A OR B.
NOT A	boolean	TRUE if A is FALSE, otherwise FALSE.
!A	boolean	Same as NOT A.

Logical Operators Example

```
hive> SELECT * FROM emp WHERE Salary>20000 && Dept=SRE;
```

Complex Operators

These operators provide an expression to access the elements of Complex Types.

Operator	Operand	Description
A[n]	A is an Array and n is an int	It returns the nth element in the array A. The first element has index 0.
M[key]	M is a Map<K, V> and key has type K	It returns the value corresponding to the key in the map.
S.x	S is a struct	It returns the x field of S.

Built-In Functions

Return Type	Signature	Description
BIGINT	round(double a)	It returns the rounded BIGINT value of the double.
BIGINT	floor(double a)	It returns the maximum BIGINT value that is equal or less than the double.
BIGINT	ceil(double a)	It returns the minimum BIGINT value that is equal or greater than the double.
double	rand(), rand(int seed)	It returns a random number that changes from row to row.
string	concat(string A, string B,...)	It returns the string resulting from concatenating B after A.
string	substr(string A, int start)	It returns the substring of A starting from start position till the end of string A.
string	substr(string A, int start, int length)	It returns the substring of A starting from start position with the given length.

Built-In Functions

string	upper(string A)	It returns the string resulting from converting all characters of A to upper case.
string	ucase(string A)	Same as above.
string	lower(string A)	It returns the string resulting from converting all characters of B to lower case.
string	lcase(string A)	Same as above.
string	trim(string A)	It returns the string resulting from trimming spaces from both ends of A.
string	ltrim(string A)	It returns the string resulting from trimming spaces from the beginning (left hand side) of A.
string	rtrim(string A)	rtrim(string A) It returns the string resulting from trimming spaces from the end (right hand side) of A.
string	regex_replace(string A, string B, string C)	It returns the string resulting from replacing all substrings in B that match the Java regular expression syntax with C.
int	size(Map<K.V>)	It returns the number of elements in the map type.
int	size(Array<T>)	It returns the number of elements in the array type.

Built-In Functions

value of <type>	cast(<expr> as <type>)	It converts the results of the expression expr to <type> e.g. cast('1' as BIGINT) converts the string '1' to it integral representation. A NULL is returned if the conversion does not succeed.
string	from_unixtime(int unixtime)	convert the number of seconds from Unix epoch (1970-01-01 00:00:00 UTC) to a string representing the timestamp of that moment in the current system time zone in the format of "1970-01-01 00:00:00"
string	to_date(string timestamp)	It returns the date part of a timestamp string: to_date("1970-01-01 00:00:00") = "1970-01-01"
int	year(string date)	It returns the year part of a date or a timestamp string: year("1970-01-01 00:00:00") = 1970, year("1970-01-01") = 1970
int	month(string date)	It returns the month part of a date or a timestamp string: month("1970-11-01 00:00:00") = 11, month("1970-11-01") = 11
int	day(string date)	It returns the day part of a date or a timestamp string: day("1970-11-01 00:00:00") = 1, day("1970-11-01") = 1

Example- Built in Functions

round() function

hive> SELECT round(2.6) from temp;

O/P:

3.0

floor() function

hive> SELECT floor(2.6) from temp;

O/P:

2.0

ceil() function

hive> SELECT ceil(2.6) from temp;

O/P:

3.0

Aggregate Functions

Return Type	Signature	Description
BIGINT	count(*), count(expr),	count(*) - Returns the total number of retrieved rows.
DOUBLE	sum(col), sum(DISTINCT col)	It returns the sum of the elements in the group or the sum of the distinct values of the column in the group.
DOUBLE	avg(col), avg(DISTINCT col)	It returns the average of the elements in the group or the average of the distinct values of the column in the group.
DOUBLE	min(col)	It returns the minimum value of the column in the group.
DOUBLE	max(col)	It returns the maximum value of the column in the group.

Example- Aggregate Functions

```
Hive> Select count(*) from emp;
```

```
Hive>Select avg(distinct price) from emp;
```

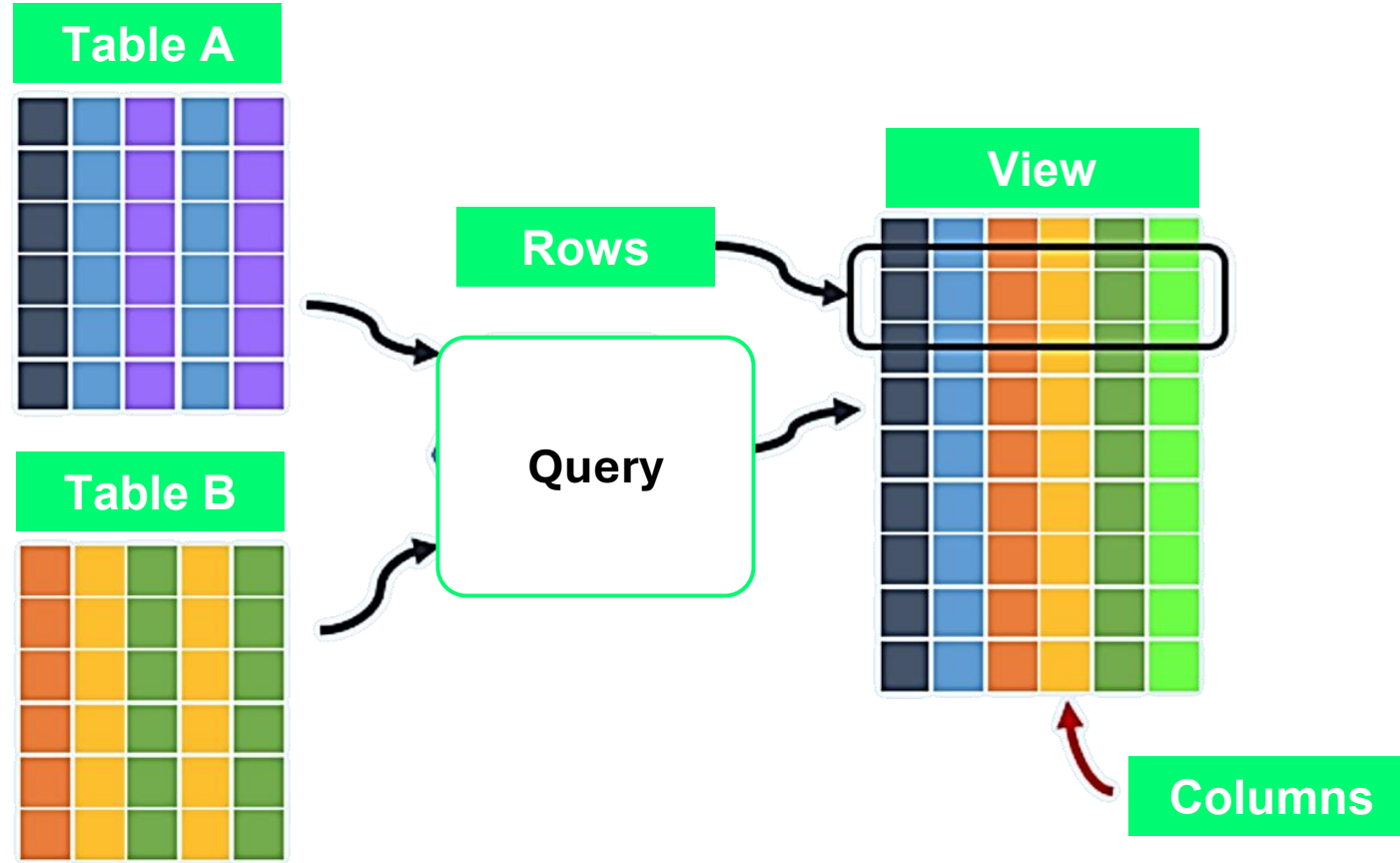
```
Hive> select max(price) from emp;
```

To count the number of distinct users by gender:

```
SELECT gender, count (DISTINCT userid)  
FROM emp  
GROUP BY gender;
```

2.11 Views

A view is purely a logical construct (an alias for a query) with no physical data behind it.



Views

- Are logical data structures that can be used to simplify queries by either hiding the complexities such as subqueries, joins, and filters or by flattening the data.
- Unlike RDBMS systems, Hive views do not store data or a physical table structure creation.
- Once the Hive view is created, its description and schema are frozen immediately.
- Any changes to the underlying tables (for example, adding a column) will not be reflected in the view's schema.
- When an underlying table is dropped or changed, subsequent attempts to query the invalid view will fail.

2.11.1 More on Hive Views

Following are the key details of Hive Views:

- Hive view is “Virtual table”
- Views used to restrict users access to subset of tables
- View is not materialized to disk when it is created. Rather, the view’s select statement is executed when the statement that refers to the view is run.
- View schema is frozen at the time of creation
- Views are read-only and may not be used directly to LOAD/INSERT/ALTER data.

2.11.2 Sample Creation of Views

The screenshot shows the command for creating a view in Hive.

```
CREATE VIEW [IF NOT EXISTS] [db_name.]view_name [(column_name [COMMENT  
column_comment], ...)]  
[COMMENT view_comment]  
[TBLPROPERTIES (property_name = property_value, ...)]  
AS SELECT ...;
```

Example:

```
CREATE VIEW emp_30000 AS  
SELECT * FROM employee  
WHERE salary>30000;
```

- If no new column names are supplied, the names of the view's columns will be derived automatically from the defining SELECT expression.
- Transformed columns will get created with _C0, _C1, etc. Names.
- A CREATE VIEW statement will fail if the view's defining SELECT expression is invalid.

Examples: Views

-- Create a view that is exactly the same as the underlying table.

```
CREATE VIEW v1 AS SELECT * FROM t1;
```

-- Create a view that includes only certain columns from the underlying table.

```
CREATE VIEW v2 AS SELECT c1, c3, c7 FROM t1;
```

-- Create a view that filters the values from the underlying table.

```
CREATE VIEW v3 AS SELECT DISTINCT c1, c3, c7 FROM t1 WHERE c1 IS NOT NULL AND c5 > 0;
```

-- Create a view that that reorders and renames columns from the underlying table.

```
CREATE VIEW v4 AS SELECT c4 AS last_name, c6 AS address, c2 AS birth_date FROM t1;
```

Examples: Views

-- Create a view that runs functions to convert or transform certain columns.

```
CREATE VIEW v5 AS SELECT c1, CAST(c3 AS STRING) c3, CONCAT(c4,c5) c5, TRIM(c6) c6,  
  "Constant" c8 FROM t1;
```

-- Create a view that hides the complexity of a view query.

```
CREATE VIEW v6 AS SELECT t1.c1, t2.c2 FROM t1 JOIN t2 ON t1.id = t2.id;
```

CHECKPOINT

Table emp (id,dept,sal)

Table emp_details(id,name)

Create view to display all details of emp;

CHECKPOINT

Table emp (id,dept,sal)

Table emp_details(id,name)

Create view to display all details of emp;

Solution:

```
CREATE VIEW view_emp AS SELECT emp.id, emp_details.name,emp.dept,emp.sal  
FROM emp JOIN emp_details ON emp.id = emp_details.id;
```

2.11.3 Alter and Drop Views

The commands for altering and drop views are given below.

Alter Views:

```
ALTER VIEW [db_name.]view_name SET TBLPROPERTIES  
table_properties;  
table_properties:  
    : (property_name = property_value, property_name =  
property_value, ...)
```

```
ALTER VIEW [db_name.]view_name AS  
select_statement;
```

Drop Views:

```
DROP VIEW [IF EXISTS] [db_name.]view_name;
```

Example

- A view is purely a logical construct (an alias for a query) with no physical data behind it, ALTER VIEW only involves changes to metadata in the metastore database, not any data files in HDFS.

Syntax:

```
ALTER VIEW [database_name.]view_name AS select_statement
```

```
ALTER VIEW [database_name.]view_name RENAME TO [database_name.]view_name
```

Examples:

```
create table t1 (x int, y int, s string);
```

```
create table t2 like t1;
```

```
create view v1 as select * from t1;
```

```
alter view v1 as select * from t2;
```

```
alter view v1 as select x, upper(s) s from t2;
```

Check View Details:

```
hive>describe formatted v1;
```

Example

- **DROP VIEW** only involves changes to metadata in the metastore database, not any data files in HDFS.
- When dropping a view referenced by other views, no warning is given, the dependent views are left as invalid and must be dropped or recreated by the user.

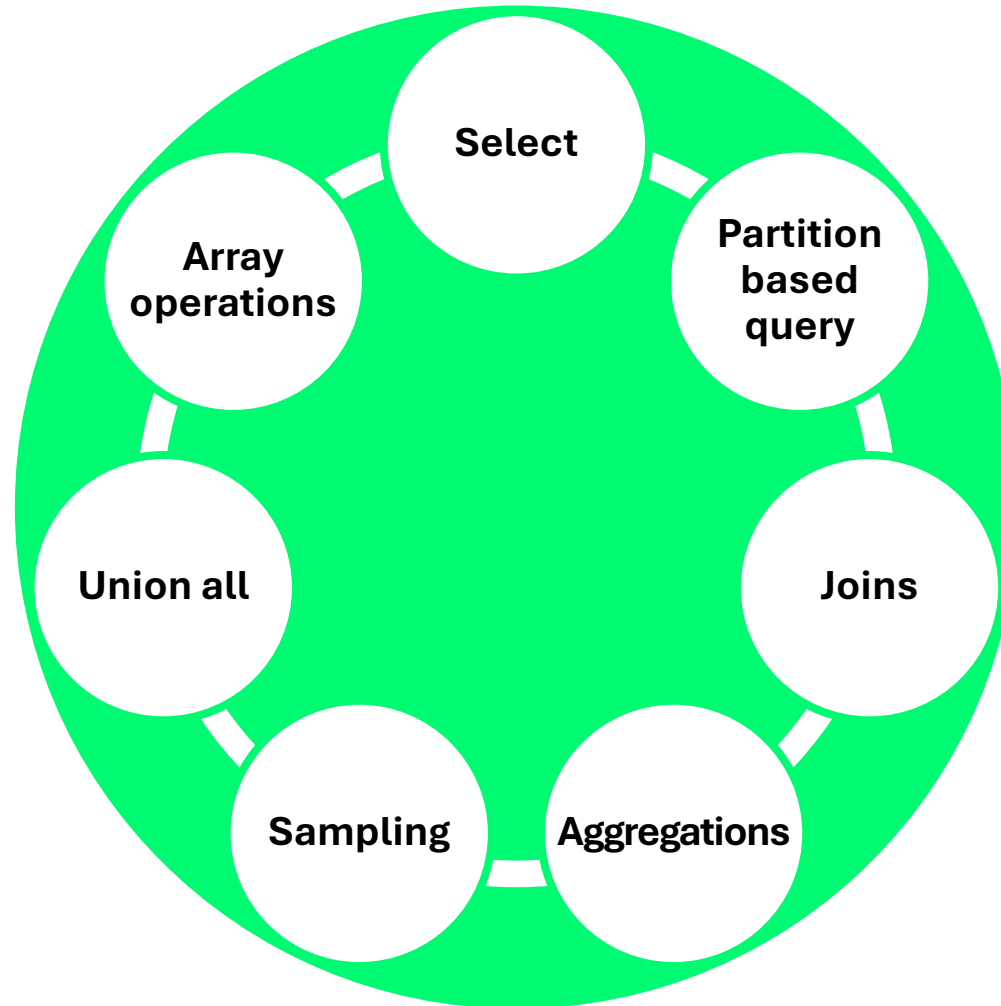
Syntax:

```
DROP VIEW [IF EXISTS] [db_name.]view_name
```

```
hive> DROP VIEW emp_30000;
```

2.12 Querying Data and Managing Output

Querying Hive data, different operations include:



Querying Data and Managing Output

- **Select:** Syntax “SELECT user.* FROM user WHERE user.active = 1;”
- **Partition based query:** What partitions to use in a query is determined automatically by the system on the basis of where clause conditions on partition columns.

Sample:

```
SELECT page_views.*  
FROM page_views  
WHERE page_views.date >= '2008-03-01' AND page_views.date <= '2008-03-31' ;
```

- **Aggregations:** Aggregations of data include, Count, Count(*), Count(distinct), Max, Min, Sum, Rank.

Sample:

```
SELECT pv_users.gender, count(DISTINCT pv_users.userid), count(DISTINCT pv_users.ip)  
FROM pv_users  
GROUP BY pv_users.gender;
```

Querying Data and Managing Output

Joins:

To fetch data from multiple tables, different types of joins can be used such as INNER, LEFT OUTER, RIGHT OUTER or FULL OUTER JOIN.

- LEFT SEMI JOIN, EQUI JOINS are also supported in Hive.

SYNTAX join_table:

```
table_reference JOIN table_factor [join_condition]
| table_reference {LEFT|RIGHT|FULL} [OUTER] JOIN table_reference
join_condition
| table_reference LEFT SEMI JOIN table_reference join_condition
| table_reference CROSS JOIN table_reference [join_condition]
```

Joins

Lets assume 2 tables:
a) Customers
b) Orders

- 1.Inner join in Hive
- 2.Left Outer Join in Hive
- 3.Right Outer Join in Hive
- 4.Full Outer Join in Hive

Customers				
ID	Name	Age	Address	Salary
1	Ross	32	Ahmedabad	2000
2	Rachel	25	Delhi	1500
3	Chandler	23	Kota	2000
4	Monika	25	Mumbai	6500
5	Mike	27	Bhopal	8500
6	Phoebe	22	MP	4500
7	Joey	24	Indore	10000

ORDERS			
OID	Date	Customer_ID	Amount
102	2016-10-08 00:00:00	3	3000
100	2016-10-08 00:00:00	3	1500
101	2016-11-20 00:00:00	2	1560
103	2015-05-20 00:00:00	4	2060

Inner Join

Fetches the records common to the both tables.

```
hive> SELECT c.ID, c.NAME, c.AGE, o.AMOUNT  
FROM CUSTOMERS c JOIN ORDERS o  
ON (c.ID = o.CUSTOMER_ID);
```

ID	NAME	AGE	AMOUNT
2	Rachel	25	1560
3	Chandler	23	3000
3	Chandler	23	1500
4	Monika	25	2060

Left Outer Join:

- Returns all the rows from the left table even though there are no matches in right table
- If ON Clause matches zero records in the right table, the joins still return a record in the result with NULL in each column from the right table

```
hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c
LEFT OUTER JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);
```

ID	Name	Amount	Date
1	Ross	NULL	NULL
2	Rachel	1560	2016-11-20 00:00:00
3	Chandler	3000	2016-10-08 00:00:00
3	Chandler	1500	2016-10-08 00:00:00
4	Monika	2060	2015-05-20 00:00:00
5	Mike	NULL	NULL
6	Phoebe	NULL	NULL
7	Joey	NULL	NULL

Right Outer Join:

- Returns all the rows from the Right table even though there are no matches in left table
- If ON Clause matches zero records in the left table, the joins still return a record in the result with NULL in each column from the left table

```
hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c RIGHT OUTER JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);
```

ID	Name	Amount	Date
3	Chandler	1300	2016-10-08 00:00:00
3	Chandler	1500	2016-10-08 00:00:00
2	Rachel	1560	2016-11-20 00:00:00
4	Monika	2060	2015-05-20 00:00:00

Note: Since all orders have valid customers, this will look similar to an INNER JOIN.

Full Outer Join

- Returns all the records from both tables and fills in NULL Values for the columns missing values matched on either side.

```
hive> SELECT c.ID, c.NAME, o.AMOUNT, o.DATE
FROM CUSTOMERS c
FULL OUTER JOIN ORDERS o
ON (c.ID = o.CUSTOMER_ID);
```

ID	NAME	AMOUNT	DATE
1	Ross	NULL	NULL
2	Rachel	1560	2016-11-20
3	Chandler	3000	2016-10-08
3	Chandler	1500	2016-10-08
4	Monika	2060	2015-05-20
5	Mike	NULL	NULL
6	Phoebe	NULL	NULL
7	Joey	NULL	NULL

This query will return:

- ✓ All customers (even those without orders)
- ✓ All orders (even those without customers - though none exist here)
- ✓ NULL values where there is no match

Querying Data and Managing Output

- **Sampling:** The sampling clause allows the users to write queries for samples of the data instead of the whole table. Currently the sampling is done on the columns that are specified in the CLUSTERED BY clause of the CREATE TABLE statement. There are two types of sampling:
 - **Bucket Sampling : e.g.**

```
SELECT * FROM T_USER_LOG_BUCKET TABLESAMPLE (BUCKET 1 OUT OF 4 AT USER_ID);
```

It will select the data from the first buckets of each partition from T_USER_LOG_BUCKET table, otherwise in normal select we can't select data in such distributed and evenly manner.

- **Block Sampling: e.g.**

```
SELECT * FROM T_USER_LOG_BUCKET TABLESAMPLE (20 PERCENT);
```

This query performs a block sample, reading approximately 20% of the HDFS storage blocks from the T_USER_LOG_BUCKET table. It returns all rows from those blocks, providing a fast but potentially non-random subset of data ideal for quick testing and exploration.

Union all

UNION is used to combine the result from multiple SELECT statements into a single result set.

Union Syntax :

```
select_statement UNION [ALL | DISTINCT]  
select_statement UNION [ALL | DISTINCT]  
select_statement ...
```

EXAMPLE:

```
SELECT key FROM src  
UNION  
SELECT key FROM src1  
ORDER BY key LIMIT 10
```

Example:

```
INSERT OVERWRITE TABLE target_table  
  SELECT name, id, category FROM source_table_1  
  UNION ALL  
  SELECT name, id, "Category159" as category FROM source_table_2
```

Nested Queries

- Subqueries in SELECT
- Subqueries in the FROM Clause
- Subqueries in the WHERE Clause

Subqueries in SELECT

- NOT SUPPORTED.
- Hive does not allow subqueries (scalar or correlated) to be used as expressions directly in the SELECT list. This is a major syntax restriction.
- Alternative: Use a JOIN (often a LEFT JOIN) in the FROM clause instead. Calculate the desired value in the join and then select it.

```
SELECT 1 + (SELECT SUM(ship_charge) FROM orders),
```

```
customer.customer_num
```

```
FROM customer;
```

Not Supported



Subqueries in the SELECT Clause

Invalid Hive Query:

```
SELECT c.customer_num,  
  
    (SELECT SUM(amount)      -- This will FAIL  
     FROM orders o  
     WHERE o.customer_num = c.customer_num)  
     AS total_amount  
FROM customer c;
```

Valid Hive Alternative (LEFT JOIN + GROUP BY):

```
SELECT  
    c.customer_num,  
    c.name,  
    SUM(o.amount) AS total_amount -- Will be NULL for customers with no orders  
FROM customer c LEFT JOIN orders o  
ON c.customer_num = o.customer_num  
GROUP BY c.customer_num, c.name;
```

Subqueries in the FROM Clause

```
SELECT ... FROM (subquery) name ...  
SELECT ... FROM (subquery) AS name ...
```

```
SELECT ... FROM ( SELECT ... FROM table_name WHERE condition ) AS subquery_name;
```

Example:

```
SELECT department, avg_salary FROM ( SELECT department, AVG(salary) AS avg_salary  
FROM employees GROUP BY department ) AS dept_summary;
```

- The subquery has to be given a name because every table in a FROM clause must have a name.
- Columns in the subquery select list must have unique names.
- The columns in the subquery select list are available in the outer query just like columns of a table.
- The subquery can also be a query expression with UNION.
- Hive supports arbitrary levels of subqueries.

The optional keyword "AS" can be included before the subquery name in Hive 0.13.

Subqueries in the FROM Clause

Simple Example:

```
SELECT col
FROM (
  SELECT a+b AS col
  FROM t1
) t2
```

Example with subquery containing a UNION ALL:

```
SELECT t3.col
FROM (
  SELECT a+b AS col
  FROM t1
  UNION ALL
  SELECT c+d AS col
  FROM t2
) t3
```

Subqueries in the WHERE Clause

- Queries where the result of the query can be treated as a constant for IN and NOT IN statements

```
SELECT *  
FROM A  
WHERE A.a IN (SELECT foo FROM B);
```

e.g.

```
SELECT * FROM employees e WHERE e.dept_id  
IN (SELECT d.dept_id FROM departments d);
```

- EXISTS and NOT EXISTS subqueries:

```
SELECT A FROM T1  
WHERE EXISTS (SELECT B FROM T2 WHERE T1.X = T2.Y)
```

e.g.

```
SELECT e.emp_id, e.emp_name FROM employees e WHERE EXISTS  
(SELECT 1 FROM employee_projects p WHERE e.emp_id = p.emp_id);
```

There are a few limitations:

- IN/NOT IN subqueries may only select a single column.
- EXISTS/NOT EXISTS must have one or more correlated predicates.
- References to the parent query are only supported in the WHERE clause of the subquery.

Limitations in subqueries in select:

- Hive supports certain subqueries in the WHERE clause.
- IN and NOT IN subqueries may select **only one column**.
- EXISTS and NOT EXISTS require **one or more correlated predicates**.
- Subqueries are supported on the **right-hand side of an expression**.
- References to the outer query are supported in the **WHERE clause of the subquery**.
- Support for subqueries in the WHERE clause was introduced in **Hive 0.13**.

```
SELECT *  
FROM customer  
WHERE customer_num IN  
(  
    SELECT DISTINCT customer_num  
    FROM orders  
    WHERE amount > 2000  
);
```

Find customers who have at least one order with an amount greater than 2000.

Checkpoint

Predict O/P?

```
SELECT stud.name,  
(SELECT dept_name  
FROM dept  
WHERE stud.id = dept.stud_id  
) AS dep_name  
FROM stud;
```

Checkpoint

Predict O/P?

```
SELECT stud.name,  
(SELECT dept_name  
FROM dept  
WHERE stud.id = dept.stud_id  
) AS dep_name  
FROM stud;
```

Error:

FAILED: ParseException line 2:5 cannot recognize input near '('
'SELECT' 'dept_name' in expression specification

Alternative:

```
Hive>SELECT stud.name,dept.dep_name  
> FROM stud  
> LEFT JOIN dept  
> on (stud.id = dept.stud_id)  
> ;
```

CHECKPOINT

1. Which clause in Hive is used to filter rows after aggregation?

- A) WHERE
- B) HAVING
- C) GROUP BY
- D) ORDER BY

2. How do you write a subquery in the FROM clause in Hive?

- A) Just write the query inside parentheses without alias
- B) Write the query inside parentheses with an alias
- C) Use WITH clause only
- D) Hive does not support subqueries

3. What does the following Hive query do?

SELECT department, COUNT(*) FROM employees GROUP BY department;

- A) Counts the total employees in the company
- B) Lists departments with employee counts
- C) Filters employees by department
- D) Shows employee salaries

4. Which keyword in Hive is used to sort the result set?

- A) ORDER BY
- B) SORT BY
- C) DISTRIBUTE BY
- D) Both A and B

5. What is required when using a subquery in the FROM clause in Hive?

- A) The subquery must select only one column
- B) The subquery must be aliased
- C) The subquery must use LIMIT clause
- D) Subqueries are not allowed in the FROM clause

1. Which clause in Hive is used to filter rows after aggregation?

- A) WHERE
- B) HAVING
- C) GROUP BY
- D) ORDER BY

Answer: B) HAVING

3. What does the following Hive query do?

SELECT department, COUNT(*) FROM employees GROUP BY department;

- A) Counts the total employees in the company
- B) Lists departments with employee counts
- C) Filters employees by department
- D) Shows employee salaries

Answer: B) Lists departments with employee counts

2. How do you write a subquery in the FROM clause in Hive?

- A) Just write the query inside parentheses without alias
- B) Write the query inside parentheses with an alias
- C) Use WITH clause only
- D) Hive does not support subqueries

Answer: B) Write the query inside parentheses with an alias

4. Which keyword in Hive is used to sort the result set?

- A) ORDER BY
- B) SORT BY
- C) DISTRIBUTE BY
- D) Both A and B

Answer: D) Both A and B

(ORDER BY sorts globally; SORT BY sorts per reducer)

5. What is required when using a subquery in the FROM clause in Hive?

- A) The subquery must select only one column
- B) The subquery must be aliased
- C) The subquery must use LIMIT clause
- D) Subqueries are not allowed in the FROM clause

Answer: B) The subquery must be aliased

Summary:

1. Apache Hive is an open source data warehouse system built on top of Hadoop cluster for querying and analyzing large datasets stored in the Hadoop file system.
2. Hive Architecture consists of Hive Clients, Hive Services, Compiler, Optimizer, Distributed Storage, etc.
3. Hive metastore is the central repository of hive metadata.
4. Hive Data Definition Language (DDL) is a subset of Hive SQL statements that are used for creating, deleting, or altering schema objects.
5. Hive partitions is a method of separating a table into related parts based on the column values like date, city, and department.
6. In Hive, views are logical data structures that can be used to simplify queries by either hiding the complexities such as subqueries, joins, and filters.